



A Python-Based Approach to Animate Geometry and Data

**Project work Submitted for the
Award of the Degree of
Bachelor of Science (Hons.) in
Mathematics**

**By
Shubham Tiwari**

Semester VIII

Department of Mathematics

Dyal Singh College

(University of Delhi)

Lodhi Road

New Delhi-110003

April 2026



A Python-Based Approach to Animate Geometry and Data

**Project work Submitted for the
Award of the Degree of
Bachelor of Science (Hons.) in
Mathematics**

**By
Shubham Tiwari**

Supervisor:

Dr. J. V. Ramani
Department of Mathematics

Co-Supervisor:

Ms. Ishver Rani Manchanda
Department of Mathematics

Department of Mathematics

Dyal Singh College

(University of Delhi)

New Delhi-110003

2026

Certificate

CERTIFICATE

I am satisfied that the project work presented by **Shubham Tiwari** entitled “**A Python-Based Approach to Animate Geometry and Data**” is worthy of consideration for the award of the degree of Undergraduate Degree with research.

I certify that:

- (a) He has pursued the prescribed course of research.
- (b) He is of good moral character.

Supervisor

Dr. J. V. Ramani
Department of Mathematics
Dyal Singh College
Email:
Mob. No:

Co-Supervisor

Ms. Ishver Rani Manchanda
Department of Mathematics
Dyal Singh College
Email:
Mob. No:

Countersigned

Head of the Department
Department of Mathematics
Dyal Singh College

Declaration

To,

The Principal
Dyal Singh College
University of Delhi
New Delhi-110003

Sir,

I have been pursuing a course of research as a student of this College and now I want to apply for submission of my project work for the award of my degree. The relevant documents are attached herewith.

Yours faithfully,

Signature: _____

Name in full: Shubham Tiwari

Dated: _____

Address: _____

Tele. No. / Mob. No.: _____

Certificate of Originality



GATHER YE THE WISDOM OF EAST AND WEST

DYAL SINGH COLLEGE

Department of Mathematics

New Delhi-110003

Certificate of Originality

Date: _____

The research work embodied in this Project entitled “**A Python-Based Approach to Animate Geometry and Data**” has been carried out by me at the Department of Mathematics, Dyal Singh College, University of Delhi, Delhi, India.

The manuscript has been subjected to plagiarism check by software. The work submitted for consideration of award of B.Sc. (Hons.) Mathematics degree is original.

Name and Signature of the Candidate

Shubham Tiwari

Plagiarism Verification Report



GATHER YE THE WISDOM OF EAST AND WEST

DYAL SINGH COLLEGE

UNIVERSITY OF DELHI

Plagiarism Verification Report

Title of the Dissertation: A Python-Based Approach to Animate Geometry and Data

Research Scholar: Shubham Tiwari

Supervisor: Dr. J. V. Ramani

Co-Supervisor: Ms. Ishver Rani Manchanda

Department: Mathematics

Faculty: Science

Date: _____

Total Pages: _____

This is to report that the above project work was scanned for similarity detection. Process and outcome are given below:

Software used: Turnitin

Date: 01-May-2026

Similarity Index: 2%

Total word count: 6768

The complete report of the above project has been forwarded to the College.

**Checked by
Name & Signature**

AI Usage Declaration

This is to declare that while AI language models were utilized as supplementary assistants—primarily to improve grammar, assist with LaTeX formatting, and debug minor Python issues—the core mathematical logic, Python code, visualizations, and intellectual property remain entirely original.

In full compliance with institutional rules, the foundational algorithms and main research findings of this report were developed independently. Furthermore, the accompanying **Turnitin** AI Writing Overview report confirms that **0%** of the qualifying text is detected as AI-generated.

Acknowledgement

I would like to express my deepest gratitude to my supervisor, Dr. J. V. Ramani, for his constant support, guidance, and encouragement throughout my project. He constantly guided me and was an invaluable mentor during this project. His extensive knowledge and support helped a lot. Through his understanding and experience, it helped me learn mathematical theory and computer programming.

I owe my gratitude to my co-supervisor, Ms. Ishver Rani Manchanda, for her insightful feedback, technical assistance, and continuous motivation during the development of this framework.

Moreover, I will also thank the Dyal Singh College's Department of Mathematics, University of Delhi, for their assistance and providing an environment conducive to academic investigation. Lastly, I thank my family and friends with a warm heart for being patient and encouraging throughout the process of bringing these mathematical animations to life.

Abstract

This project presents a strong computational framework in Python designed to convert static geometric equations and mathematical data into dynamic and interactive animations. Students should focus on visualizing abstract mathematical concepts in their thinking, making theoretical concepts easier to visualize and understand.

Through the use of Python's powerful scientific ecosystem, the project employs NumPy for vectorized arrays, heavy data processing, and linear algebra. It creates a standardized pipeline for programmatic animation. The implementation proceeds from basic 2D kinematics, like parametric curves and random movements, to sophisticated 3D shapes. The use of $SO(3)$ rotation matrices reveals the topological morphing of L_p norm spaces and acts as a catalyst in the implementation of differential geometry concepts. These include minimal surfaces such as the catenoid, helicoid, and Boy's surface.

The project demonstrates that by converting continuous mathematical domains, appropriate programmatic animation may increase student engagement. We should relate mathematical concepts to student experience as far as possible to improve the understanding of mathematics.

Contents

Certificate	i
Declaration	ii
Certificate of Originality	iii
Plagiarism Verification Report	iv
AI Usage Declaration	vi
Acknowledgement	vii
Abstract	viii
List of Symbols / Notation	xiv
1 Introduction	1
1.1 Project Motivation	1
1.2 The Mathematics Visualization Problem	1
1.3 The Importance of Animation in Education	2
1.4 Why Python?	3
2 Literature Review	4
2.1 Previous Work in Mathematical Visualisation	4
2.2 The Python Animation Ecosystem	4
2.3 Challenges in Learning Geometry	5
2.4 Research on Educational Animation	5
3 Project Aim and Objectives	6
3.1 Main Objective	6
3.2 Specific Goals	6
3.3 Learning Outcomes	7
4 Mathematical Foundations	8
4.1 Norms and Metric Spaces	8
4.2 Coordinate Geometry and 2D Parametrization	8
4.2.1 Fundamental 2D Parametric Equations	9

4.2.2	The Sliding Ladder Constraint	9
4.3	Linear Algebra and Spatial Motion	10
4.3.1	Rigid Motions and Orthogonal Matrices	10
4.4	Differential Geometry of 3D Surfaces	11
4.5	Projection and Viewpoint Metrics	12
5	Methodology	13
5.1	The Implementation Pipeline	13
5.2	Discretisation: From Theory to Arrays	14
5.3	Animation Engine Workflow	14
6	Technologies Used	16
6.1	Core Development Environment	16
6.2	Core Development Environment	16
6.3	The Python Libraries	17
6.4	Environment Management: uv	17
7	Repository Structure	19
7.1	Code Organization	19
8	Implementation	21
8.1	The General Animation Architecture	21
8.2	2D Animation Implementation	21
8.3	3D Animation Implementation	23
9	Results and Analysis	26
9.1	Kinematic Geometry: The Sliding Ladder	26
9.2	Visualizing Metric Spaces: L_p Norms	27
9.3	3D Surface Geometry and Rigid Body Dynamics	28
9.3.1	Fundamental Rigid Bodies	28
9.3.2	Complex and Minimal Surfaces	30
9.3.3	Dynamic Systems and Transformations	30
10	Challenges Faced	32
10.1	Rendering and Performance Issues	32
10.2	Mathematical Conversion and Scaling	32
10.3	Environment and Dependency Bugs	33
11	Conclusion and Future Scope	34
11.1	Summary of Achievements	34
11.2	Current Limitations	34

11.3 Future Scope	35
References	36
A Open-Source Code & GitHub Repository	37
B Proof of Publication	38
C Conference Presentation Certificate	40
D Presentation Slides	41
E Research Progress & Monthly Reports	42
F Extended Code Snippets	44
F.1 3D Rigid Body Engine: Rotating Cube	44
F.2 Multi-Axis Spherical Rotations	47
F.3 Topological Geometry: The Cutaway Torus	50
F.4 Differential Topology: Boy's Surface	52
F.5 Combined Kinematics: Translation and Rotation	54
F.6 Articulated Kinematics: Human Motion Simulation	56
F.7 Stochastic Processes: 2D Random Walk	59
F.8 Metric Space Topology: L_p Norm Unit Balls	61
G Plagiarism Originality Report	63
H AI Writing Overview	66

List of Figures

5.1	The Computational Animation Pipeline	13
5.2	Visual diagram of the animation loop.	15
6.1	The core software stack utilised for project development.	16
6.2	The core Python library pipeline.	17
6.3	Project setup and execution workflow using uv.	18
7.1	Project Structure on <i>GITHUB</i>	20
9.1	The Cat on the Sliding Ladder: A visual progression over time.	27
9.2	Visual progression of the l_p unit ball as p increases.	28
9.3	Fundamental geometric rigid body rotations.	29
9.4	Visualization of advanced differential geometry.	30
9.5	Dynamic multi-body systems and topological transformations.	31
9.6	Combining multiple geometric arrays to simulate human kinematic motion.	31
A.1	The project’s official GitHub repository landing page.	37
B.1	Official publication acceptance letter from AkiNik Publications (Ref. No.: MATH-35-02).	39
C.1	Official Certificate of Paper Presentation at SCA-2026.	40
D.1	Core slides from the official project presentation.	41
E.1	Monthly progress review presentations delivered during Semester VIII.	43

List of Tables

4.1	Parametric Equations of Fundamental 2D Shapes	9
4.2	Parametric Equations of Advanced 3D Surfaces	12
5.1	Translating continuous mathematics into discrete vector operations. . .	14

List of Symbols / Notation

x, y, z	Cartesian Coordinates
θ, ϕ	Angular parameters (Azimuth, Elevation)
$R_z(\theta)$	Rotation matrix about the z-axis
$ x _p$	L_p Norm
$\mathbb{R}^3$	3-Dimensional Euclidean Space

Chapter 1

Introduction

1.1 Project Motivation

Since childhood, we have all been learning Mathematics—from basic counting and drawing shapes to addition, multiplication, and tables. Remember how our parents and teachers taught us these topics? They would often say, “Beta, assume we have 10 ice creams. If you eat 4 of them, how many are left?” We would then count on our fingers and answer, “six.” Our creative minds would naturally visualise that scenario to find the answer. As we grew older, we tried to learn and visualise more. However, it became harder to understand advanced mathematical concepts, as topics like trigonometry often started going above our heads. There are many reasons why we gradually lose interest in Mathematics—such as teachers not presenting the subject engagingly, or students becoming bored during lectures and looking outside the classroom window. Whatever the reason may be, one thing is clear: we slowly lose interest in the subject.

But what if we started including animations and visualisation in Mathematics? What if we could see how derivatives work, how integration works, and how areas are broken down into small rectangular pieces and then combined again (of course, by taking the width of each rectangle close to zero) to obtain the final area? What if we could see how mathematical functions actually work—like machines that take input from one side and produce output from the other? What if we could visualise how a function’s domain looks and how its codomain appears?

What if we could actually see all the things we are learning? One may ask, “Is it possible to visualise all of Mathematics?” Of course, this question naturally arises because Mathematics is often considered highly abstract in nature.

1.2 The Mathematics Visualization Problem

The answer to the previous question is **YES!** Mathematics can be visualised because, in many ways, it reflects patterns found in nature. I remember a quote from one of the brilliant mathematicians and *Fields Medal* winners, **William Thurston**:

Quote

“I think that the most important thing a mathematician can do is to show that mathematics is a human activity, not some supernatural fictional world, and that it is one of the most creative fields.”

This is what many students are unable to experience. We learn theorems and memorise proofs, but if someone asks about the intuition or logic behind them, students often remain silent because they lack a strong conceptual understanding of the topic. This is where visualisation becomes important.

For example, there is a beautiful subject called Differential Geometry, which studies curves and surfaces. In many cases, teachers ask students to memorise parametrisation equations for these surfaces. However, I believe the most important aspect is to actually see how these shapes are formed—how mathematics and coding work together to create visual structures and bring these shapes to life.

Thus, the gap between theory and building strong intuition is what this project is trying to fill.

1.3 The Importance of Animation in Education

Animations of math concepts make learning more fun and interactive. I have taught students in tuition classes and once tried to explain trigonometry using only theoretical explanations. However, they had trouble understanding and often said, “Sir, kuch samajh nahi aa raha, kya yeh itna hard subject hai?” I realized that this was the wrong way to go because students may lose interest in the subject if they don’t build intuition.

I didn’t just tell them that $\sin \theta = \frac{P}{H}$, I showed them a triangle and let them try out different side lengths. By looking and exploring, they found that there was always a constant relationship. I then told them that this constant relationship is what we call sine. After they saw the graph of the sine function, they started to understand how sine and other trigonometric ratios repeat.

This shows how using animation and visualization can make learning faster, clearer, and more fun.

In today’s rapidly changing world, a traditional approach won’t work easily. Teachers must upgrade their teaching methods, and students should also make an effort to develop a deeper understanding by visualizing concepts. When these students become teachers themselves, they can pass this knowledge more interactively and intuitively. This helps create genuine learners rather than students who simply memorize information like parrots.

1.4 Why Python?

One of the genuine question arises: why not we go and use built in software for mathematical animation such as **Geogebra**, I say its good, but learning the process from scratch, gives you different feeling, and also note that these software have only have finite set of animations, and if someone ask why not **C++**, or any other language, the reason is simple!, The easiness of **Python**, is what makes it great that even non-programmers would also start doing coding, after learning that language.

The open source, code readability, and easier error detection is what makes python a great choice for this task, because you have to only focus on core logic and mathematical idea, not much care about the programming part, Python minimal nature will handle this.

Its massive and Math heavy library like **numpy**, **Matplotlib**, **Seaborn**, **Scipy**, **Sympy**, **Plotly** etc., will help us to create animations in the easiest way possible. One of the biggest advantages of Python is its simple and readable syn- tax. For example, plotting a mathematical function requires only a few lines of code:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-10, 10, 500)
5 y = np.sin(x)
6
7 plt.plot(x, y)
8 plt.title("Graph_of_sine_function")
9 plt.xlabel("x-axis")
10 plt.ylabel("y-axis")
11 plt.grid(True)
12 plt.show()
```

This example shows how easily Python can transform a mathematical equation into a visual graph. Such simplicity makes Python highly suitable for mathematical exploration, experimentation, and educational visuali- sation.

Chapter 2

Literature Review

2.1 Previous Work in Mathematical Visualisation

After the COVID-19 era, there is a massive change in education. Many students start exploring the online world first time, platforms like Youtube gets boom due to this, students realise that there are more ways to study than just reading a whole book, you can see in the real world what was written in that book, educators start using various visualisation tools to teach students.

Most tools, like built-in software such as **GeoGebra** and **Desmos**, and programming languages like *Matlab*, *Maple*, *Mathematica*, are excellent in creating visualization, but full-fledged animation is still not possible there; also, cost is a big issue, as they were often expensive or inaccessible to most of the students.

So the point is that there is a huge gap between full fledged control animation and just drag and drop, and that is why learning to create those animations is crucial.

2.2 The Python Animation Ecosystem

As discussed earlier, Python has one of the best ecosystems for this, and why I am supporting it has various reasons.

1. Open-source nature
2. Readable documentation
3. Massive community support
4. Animations are smooth and highly controllable
5. Libraries written in C++, like *NumPy* and *Matplotlib*, make animation execution super fast

The library that we mostly use here is *Matplotlib*. Initially, it was just a static plotting library, but with time, the introduction of the animation module, especially ***FuncAnimation***, made it a very powerful engine and an excellent pipeline for dynamic visuals.

Although one can ask why not use *Manim*, the reason is simple: it is not easily accessible and has uneven documentation. Although it popularised programming-based mathematical visualisation, *Matplotlib*'s focus on data plotting and geometric foundations makes it excellent for animations.

2.3 Challenges in Learning Geometry

One of the problems I see is that students sometimes easily visualise 2D by drawing it on paper and using other methods, but they scratch their heads when it comes to 3D visualisation, since the z -axis visualisation is not as simple as it looks.

Students can easily solve equations and convert them from implicit or explicit form to parameterised form, like $x(\theta) = a \cos(\theta)$ and $y(\theta) = b \sin(\theta)$, but they face problems understanding how that angle changes with time and how that graph is actually plotted.

Most linear algebra concepts, like the rotation matrix and $R_z(\theta)$, feel abstract and seem like arbitrary numbers until you actually see an object rotating using this simple matrix.

2.4 Research on Educational Animation

There are various research studies showing that visualisation supports and extends someone's knowledge. One of the most important theories is the Cognitive Theory of Multimedia Learning [1]. It points out that people learn more deeply from words and pictures combined rather than from words alone.

This theory suggests that learners actively organise visual and verbal information and prepare a mental model, which is otherwise constrained by limited working memory capacity [1]. Well-placed animation prevents cognitive overload; by drawing and understanding a shape or concept step by step, a student's brain can sequentially understand complex concepts.

This educational research indicates that moving visuals increase student engagement, improve focus, and support long-term memory retention of complex and heavy mathematical concepts.

Chapter 3

Project Aim and Objectives

3.1 Main Objective

The central idea of this project can be summarised as follows:

“To develop a robust, Python-based computational framework which is capable of translating the static geometric equations and data into dynamic and interactive animations, thereby enhancing the students’ understanding and educational perception of abstract mathematical spatial transformations.”

3.2 Specific Goals

To achieve the main aim of this project, it was divided into the following specific, practical and feasible goals:

- **2D Mathematical Animation:** We start with a 2D implementation, covering the basics of creating a canvas, placing shapes on it, moving those shapes, how 2D graphs are drawn, and how they evolve or change over time.
- **3D Surface Generation & Animation:** Then we extend our knowledge to the third dimension and see how just adding one more axis enhances our understanding of real-world objects and some shapes of differential geometry (like **Minimal Surfaces**), what’s their application is like, and also some 3d problems.
- **Mathematical Translation:** To convert implicit and explicit geometric equations into parametric forms so that they can be evaluated over discrete time steps for animation!
- **Technical Implementation:** To get the advantage of Python’s Scientific ecosystem, especially **Numpy** for heavy matrix multiplication and other matrix operations (like rotation matrix and vector translations), and **Matplotlib** for rendering the animation frame-by-frame.

3.3 Learning Outcomes

Ok, so what can one gain from this project?

I think the execution of this project will yield numerous key learning outcomes. First, it will bridge the gap between theoretical mathematical constructs and practical computational implementation. Through the development of this animation framework, the following things can be achieved:

- **Mathematical Parametrisation:** One could gain a robust understanding of converting static and implicit equations into time-dependent parametric equations that are suitable for frame-by-frame animation.
- **Computational Linear Algebra:** Achieve mastery over rotation matrices such as $R_z(\theta)$, translation vectors, and coordinate transformations, along with understanding how to apply them to move objects in real 3D space.
- **Scientific Computing Proficiency:** Develop expertise in the Python scientific collection, specifically using NumPy for vectorised calculations and Matplotlib as the core animation engine for dynamic visualisation.
- **Spatial Reasoning:** Enhance the ability to visualise complex 3D geometric properties—such as azimuth and elevation—which are important in differential geometry, computer graphics, and computer vision.
- **Reproducible Research:** Learn to manage project dependencies and virtual environments efficiently using the `uv` package manager to ensure code portability and reproducibility.

Chapter 4

Mathematical Foundations

Programmatic animation is the continuous evaluation of geometric systems over discrete intervals. As established in the project’s framework, dynamic visualisation can be defined by the fusion of three fundamental elements:

$$\text{Animation} = \text{Geometry} + \text{Transformation} + \text{Time} \quad (4.1)$$

This chapter details the mathematical machinery—ranging from functional analysis to differential geometry and linear algebra—required to translate static equations into evolving mathematical animations.

4.1 Norms and Metric Spaces

Before defining shapes and transformations, it is necessary to formally define the space in which these animations occur. Our computational environment is modeled on $\mathbb{R}^n$ (specifically $\mathbb{R}^2$ and $\mathbb{R}^3$), which is a complete metric space governed by distance functions known as norms.

According to functional analysis principles [8], the general L_p norm for a vector $x = (x_1, x_2, \dots, x_n)$ in $\mathbb{R}^n$ is defined as:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad 1 \leq p < \infty \quad (4.2)$$

For geometric rendering and physics simulations in this project, we strictly utilize the L_2 Euclidean norm ($p = 2$). This norm is essential because it is induced by the standard inner product, which preserves the concepts of angles and orthogonality during spatial transformations [8].

4.2 Coordinate Geometry and 2D Parametrization

Classical coordinate geometry often describes shapes using implicit equations of the form $F(x, y) = 0$ [7]. However, for computational animation, these shapes must be

represented as time-dependent vector functions. This is achieved through *parametrization*, where spatial coordinates are expressed as functions of an independent parameter $t \in \mathbb{R}$ or an angle θ .

4.2.1 Fundamental 2D Parametric Equations

Table 4.1 outlines the standard parametric representations utilized in the 2D animation engine of this project, derived from standard coordinate geometry [7].

Table 4.1: Parametric Equations of Fundamental 2D Shapes

Shape	Implicit Eq.	Parametric Eq.	Geometric Significance
Line	$Ax + By + C = 0$	$x(t) = x_0 + d_x t$ $y(t) = y_0 + d_y t$	Describes a continuous linear locus from an initial vector extending along a direction vector.
Circle	$x^2 + y^2 = r^2$	$x(\theta) = r \cos(\theta)$ $y(\theta) = r \sin(\theta)$	Generates a symmetric periodic locus where the radius vector maintains a constant L_2 norm.
Ellipse	$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$	$x(\theta) = a \cos(\theta)$ $y(\theta) = b \sin(\theta)$	Scales the circular domain anisotropically. The semi-axes a and b drive orbital mechanics.
Parabola	$y^2 = 4ax$	$x(t) = at^2$ $y(t) = 2at$	Represents kinematic projectile trajectories and quadratic growth paths over continuous time t .
Hyperbola	$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$	$x(t) = a \cosh(t)$ $y(t) = b \sinh(t)$	Utilizes hyperbolic trigonometric functions to draw outward branching asymptotes.

4.2.2 The Sliding Ladder Constraint

A prime example of dynamic geometric constraint in this project is the classical sliding ladder problem [7]. A ladder of fixed length L slides against orthogonal axes. The constraint is defined by the Pythagorean metric $x^2 + y^2 = L^2$. In our animation loop,

the motion is simulated by systematically decrementing the angle θ :

$$P(\theta) = \begin{bmatrix} L \cos \theta \\ L \sin \theta \end{bmatrix}, \quad \text{for } \theta \in \left[\frac{\pi}{2}, 0\right] \quad (4.3)$$

4.3 Linear Algebra and Spatial Motion

To animate rigid motion in $\mathbb{R}^3$, linear transformations must be applied. For any position vector P , the transformed coordinate P' is calculated via affine transformation:

$$P' = RP + T \quad (4.4)$$

where $T \in \mathbb{R}^3$ represents the translation vector (governing linear displacement), and R is a rotation matrix [9].

4.3.1 Rigid Motions and Orthogonal Matrices

For a transformation to look realistic and preserve the shape of an object, it must be a rigid motion. Mathematically, this means R must belong to the Special Orthogonal Group, denoted as $SO(3)$ [9]. A matrix $R \in SO(3)$ has two critical properties:

1. $R^T R = I$ (It is orthogonal, preserving the inner product and L_2 norm distances).
2. $\det(R) = 1$ (It preserves orientation, preventing the object from flipping inside out).

To formally demonstrate that spatial transformations in our animation framework do not distort the geometric objects, we must prove that the L_2 norm (length) of any position vector is invariant under multiplication by an orthogonal rotation matrix R [9].

Proof of Norm Preservation:

Let $P \in \mathbb{R}^3$ be a position vector, represented as a column matrix. The squared L_2 norm of P is algebraically equivalent to its inner (dot) product with itself:

$$||P||^2 = P^T P \quad (4.5)$$

Let P' be the new position vector after applying a rotation matrix $R \in SO(3)$, such that $P' = RP$. The squared norm of the transformed vector P' is evaluated as follows [9]:

$$||P'||^2 = (P')^T P' = (RP)^T (RP) \quad (4.6)$$

By applying the standard matrix property of transposes, $(AB)^T = B^T A^T$, we expand

the expression:

$$||P'||^2 = P^T R^T R P \quad (4.7)$$

Since R is an orthogonal matrix by definition, its transpose is its inverse. Therefore, $R^T R = I$, where I is the 3×3 identity matrix [9]. Substituting this property into our equation yields:

$$||P'||^2 = P^T (I) P = P^T P = ||P||^2 \quad (4.8)$$

Taking the principal square root of both sides concludes the proof:

$$||P'|| = ||P|| \quad (4.9)$$

This algebraic guarantee is fundamentally why our Python framework can continuously multiply vertex arrays by rotation matrices, frame after frame, without the 3D objects expanding, shrinking, or warping over time.

Rotations about the standard Cartesian axes are governed by the following Euler angle matrices:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \quad (4.10)$$

$$R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \quad (4.11)$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (4.12)$$

4.4 Differential Geometry of 3D Surfaces

While a curve is defined by a single parameter mapping $\mathbb{R} \rightarrow \mathbb{R}^3$, generating a continuous surface requires a bivariate parametric mapping $S(u, v)$ from a 2D domain to a 3D manifold [10].

$$S(u, v) = \begin{bmatrix} f(u, v) \\ g(u, v) \\ h(u, v) \end{bmatrix}, \quad \text{where } (u, v) \in D \subset \mathbb{R}^2 \quad (4.13)$$

Table 4.2 outlines the complex 3D manifolds programmed within this computational framework, including classic spheres and minimal surfaces. Minimal surfaces (like the

Catenoid and Helicoid) are highly significant in differential geometry as they possess a mean curvature of zero ($H = 0$) at all points [10].

Table 4.2: Parametric Equations of Advanced 3D Surfaces

Surface	Parametric Equations	Geometric Explanation
Sphere	$x(u, v) = r \sin(u) \cos(v)$ $y(u, v) = r \sin(u) \sin(v)$ $z(u, v) = r \cos(u)$	The locus of all points at distance r from the origin. Here, u acts as the polar angle and v as the azimuthal angle.
Torus	$x(u, v) = (R + r \cos v) \cos u$ $y(u, v) = (R + r \cos v) \sin u$ $z(u, v) = r \sin v$	Generated by revolving a circle of radius r in 3D space about a coplanar axis at distance R from the center [10].
Catenoid	$x(u, v) = a \cosh(v) \cos(u)$ $y(u, v) = a \cosh(v) \sin(u)$ $z(u, v) = av$	A minimal surface ($H = 0$) formed by rotating a catenary curve. It mathematically minimizes surface area for given circular boundaries.
Helicoid	$x(u, v) = u \cos v$ $y(u, v) = u \sin v$ $z(u, v) = cv$	A ruled minimal surface shaped like a spiral ramp. It is the only ruled minimal surface other than the flat plane [10].

4.5 Projection and Viewpoint Metrics

A core mathematical challenge in computational graphics is converting 3D data into a 2D projection on a computer screen. When utilizing the standard 3D projection provided by `Matplotlib`, the simulated camera perspective is determined by two spherical coordinate angles:

- **Azimuth (ϕ):** The horizontal angle of rotation around the z -axis.
- **Elevation (θ):** The vertical angle tracking the altitude above the base $x - y$ plane.

By programmatically looping these parameters with a constant differential per frame, a continuous orbital rotation around the geometry is achieved. For custom rendering, a perspective scaling factor inversely proportional to distance ($\text{Scale} \propto \frac{1}{z}$) is applied to project coordinates dynamically onto a flat visual plane.

Chapter 5

Methodology

5.1 The Implementation Pipeline

Now, for making mathematical animation, we follow a strict four-step computational pipeline. Every geometry problem, shapes, surfaces, from a simple 2D random walk to a complex 3D minimal surface like helicoid, catenoid, etc., passes through these stages sequentially. To illustrate this architecture, the entire pipeline is mapped out below:

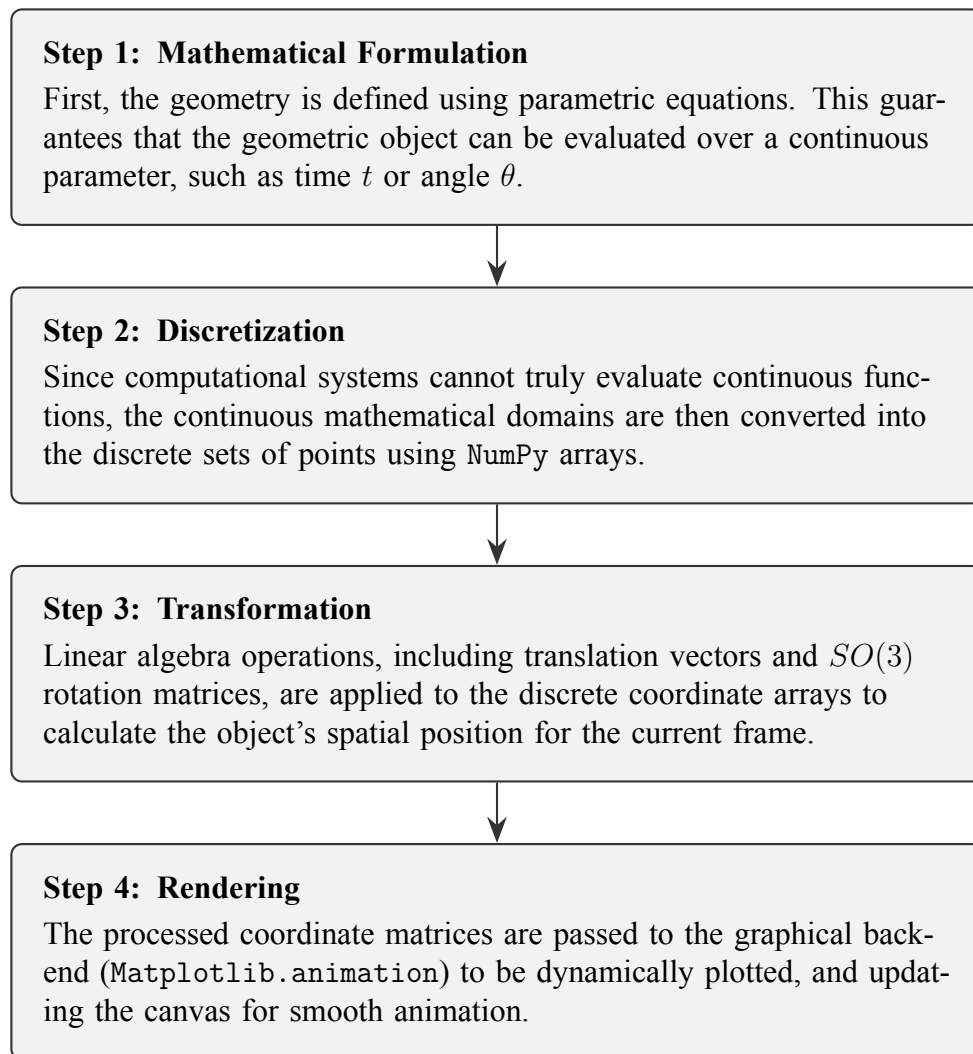


Figure 5.1: The Computational Animation Pipeline

5.2 Discretisation: From Theory to Arrays

Computers cannot render infinity truly. To animate a continuous geometric curve, it must be mathematically "sliced" into hundreds of discrete, computational frames. This process, known as discretisation, is powered entirely by the NumPy library.

Instead of evaluating points one by one using computationally expensive loops, the animation engine takes advantage of **Vectorisation**. By applying mathematical functions to entire arrays simultaneously, rendering times are drastically reduced, ensuring a smooth frame rate. To simulate a continuous curve, we generate a linearly spaced array of values using functions such as `np.linspace`¹.

Table 5.1 illustrates how continuous mathematical concepts are translated directly into discrete computational logic within the pipeline.

Mathematical Concept			Python / NumPy Equivalent
Continuous	Domain	→	<code>theta = np.linspace(0, 2*np.pi, frames)</code>
Parametric	Vectorization	→	<code>x = radius * np.cos(theta)</code>
Matrix Multiplication ($R \cdot P$)		→	<code>P_transformed = np.dot(R_matrix, P_array)</code>

Table 5.1: Translating continuous mathematics into discrete vector operations.

5.3 Animation Engine Workflow

To build the animation, we move data through a simple loop. We turn pure math into code, frame by frame. The entire engine logic is shown below using mathematical steps and arrows.

$$t \in [0, t_{max}]$$

(1. Define Time or Angle Input)



¹The `np.linspace(start, stop, num)` function generates `num` evenly spaced numerical samples over the closed interval `[start, stop]`, giving exact control over the total number of animation frames [4].

$$P(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

(2. Generate Parametric Geometry)

$\Downarrow$

$$P' = R \cdot P(t) + T$$

(3. Apply Matrix Transformation)

$\Downarrow$

$$\text{Canvas} \leftarrow P'$$

(4. Render to Matplotlib Canvas)

$\circlearrowleft$ *(Repeat Loop for Next Frame)*

This loop runs continuously inside the `update(frame)` function. By calculating P' for every small step of t , the geometry comes to life on the screen.

Figure 5.2: Visual diagram of the animation loop.

Chapter 6

Technologies Used

Now, building a smooth animation engine requires the right choice of tools. Instead of writing everything from scratch, we prefer to use and leverage a modern and open-source software stack. This chapter exactly outlines the language and tools, libraries and environment tools that are used to develop this project.

6.1 Core Development Environment

To ensure the code is reliable and easy to test, the project was built using the following core technologies:

6.2 Core Development Environment

To ensure the code is reliable, reproducible, and easy to test, the project was built using a modern, interconnected software stack. The primary tools are summarised below:

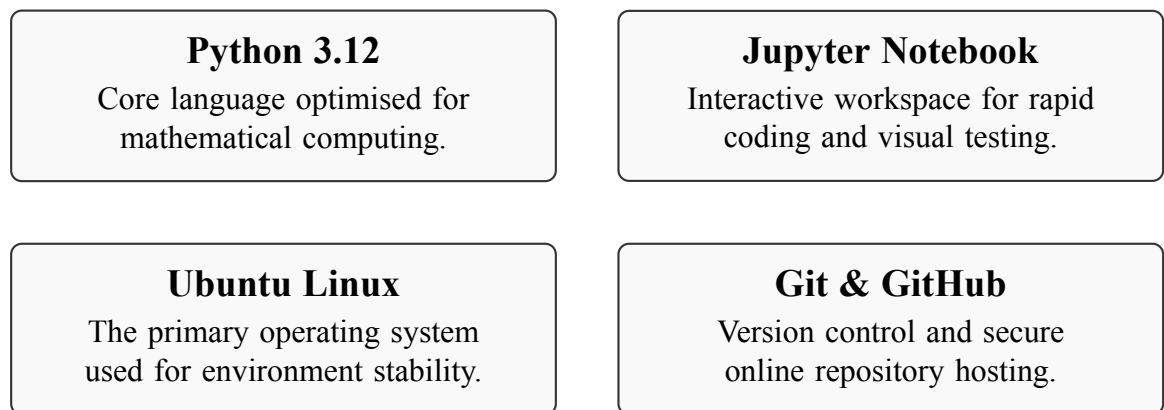


Figure 6.1: The core software stack utilised for project development.

6.3 The Python Libraries

Rather than writing complex rendering algorithms from scratch, this project relies on two foundational Python libraries. They work together in a continuous, high-speed pipeline: NumPy processes the raw numbers, and Matplotlib paints them onto the screen.

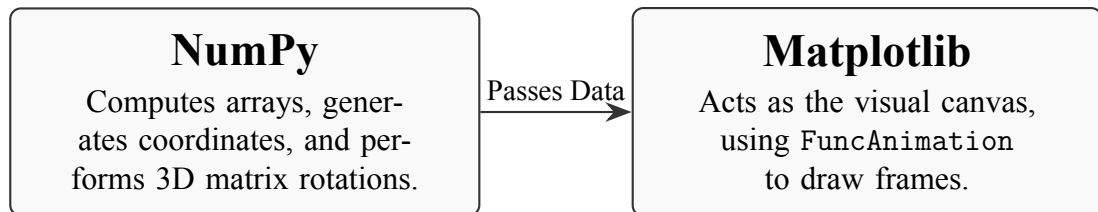


Figure 6.2: The core Python library pipeline.

6.4 Environment Management: uv

Managing Python libraries manually often leads to “dependency hell”—where code works on one computer but breaks on another due to version mismatches.

To solve this, the project uses `uv`, an ultra-fast project manager written in Rust.[5] It guarantees that the code will run flawlessly on any machine by automatically isolating and installing the exact required dependencies. The setup process is entirely streamlined into four simple terminal commands:

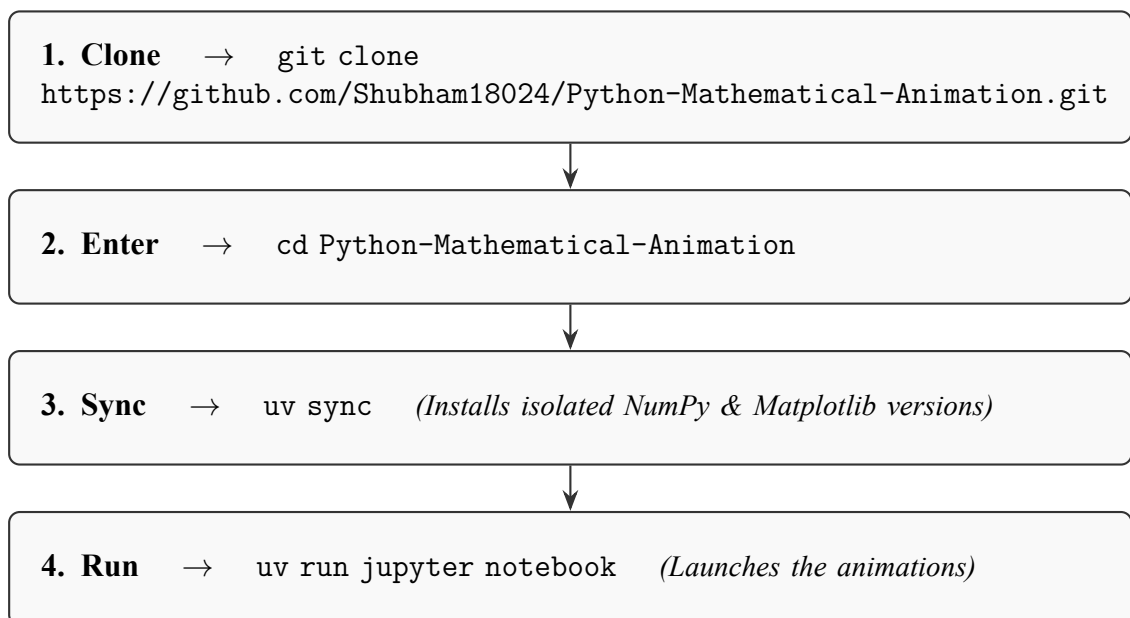


Figure 6.3: Project setup and execution workflow using uv.

Chapter 7

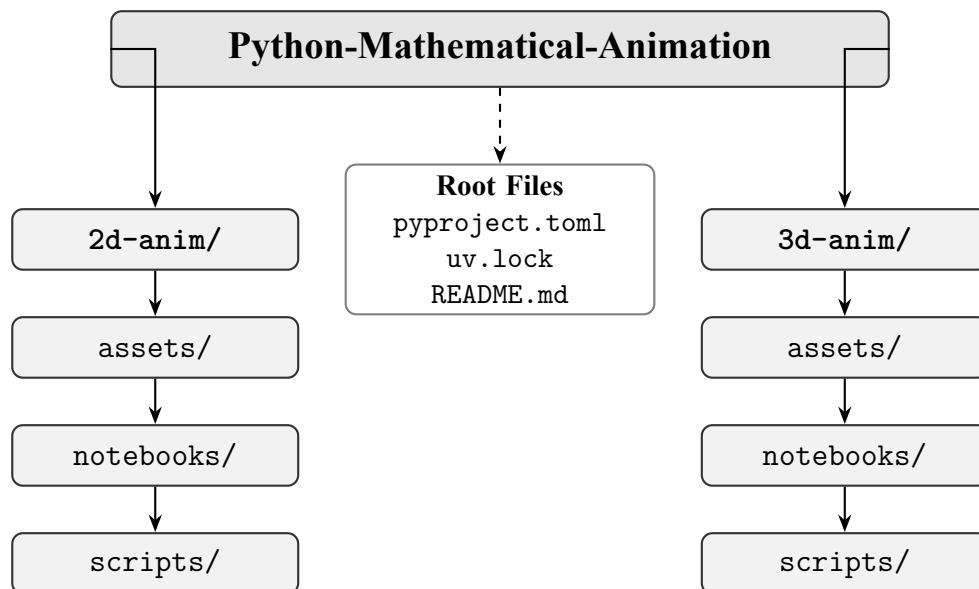
Repository Structure

This complex computational project requires a strict organisational architecture to remain maintainable and scalable. The code and animation files are organised into a modular structure, separating visual assets, interactive notebooks, and core execution scripts.

7.1 Code Organization

The project repository is partitioned into two primary domain directories: `2d-anim` and `3d-anim`. Within these directories, the workflow is further divided by file type and execution stage.

- **assets/:** Stores the final rendered outputs, including `.mp4` video files and `.gif` animations.
- **notebooks/:** Contains `.ipynb` Jupyter Notebooks utilized for prototyping mathematical logic and rapid interactive testing.
- **scripts/:** Houses the final, production-ready `.py` files containing the structured `update()` functions for the `FuncAnimation` engine.

Figure 7.1: Project Structure on *GITHUB*

Chapter 8

Implementation

Now, in this chapter, we give the details about the implementation, or we say the execution of Python code. The implementation of Python code is uniform across the whole scripts and notebooks, and the utilisation of Matplotlib's `FuncAnimation` class to evaluate parametric equations over time.

Instead of giving the whole project's source code, here we just provide the general structure of the code. This chapter isolates the general structure logic required to generate both 2D curves and 3D rotating myriad.

8.1 The General Animation Architecture

Every animation script in this project adheres to a standardized architectural blueprint. The workflow strictly separates static environmental setup from dynamic mathematical evaluation.

The universal structure consists of:

1. **Setup Phase:** Initializing the graphical canvas, defining limits, and generating the discretized numerical arrays (e.g., using `np.linspace`).
2. **Mathematical Definitions:** Programming the parametric equations or establishing the rotation/translation matrices.
3. **Initialization Routine (`init`):** A function that renders the first frame or empty axes to optimize rendering overhead.
4. **Update Routine (`update`):** The core engine function that receives a frame variable, applies mathematical transformations, and updates the canvas data.

8.2 2D Animation Implementation

The following snippet demonstrates the generalized code structure used to animate 2D kinematics. This specific example visualizes a dynamic sinusoidal wave whose amplitude evolves over time (frames).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 # 1. SETUP AXES AND ENVIRONMENT
6 fig, axs = plt.subplots(figsize=(6,6))
7 axs.set_xlim(-10, 10)
8 axs.set_ylim(-10, 10)
9 axs.set_aspect('equal', adjustable='box') # Preserve geometric
    aspect ratio
10 axs.grid(True)
11
12 # 2. DISCRETIZE THE DOMAIN
13 u = np.linspace(-10, 10, 100)          # Spatial domain for the
    curve
14 t = np.linspace(-10, 10, 100)          # Time/Frame parameter
    controlling amplitude
15
16 # Initialize empty line object to be updated continuously
17 straight_line, = axs.plot([], [], linestyle='-', color='blue',
    linewidth=3)
18
19 # 3. INITIALIZATION FUNCTION
20 def init():
21     straight_line.set_data([], [])
22     return straight_line,
23
24 # 4. UPDATE FUNCTION (THE CORE ENGINE)
25 def update(frame):
26     # 'frame' corresponds to the values passed from 't'
27     k = frame
28
29     # Evaluate parametric y-coordinates dynamically:  $y = k * \sin(u)$ 
30     y_evaluated = k * np.sin(u)
31
32     # Inject newly calculated coordinates back into the plot
    object
33     straight_line.set_data(u, y_evaluated)
34     return straight_line,
35

```

```

36 # 5. EXECUTE FUNCANIMATION
37 ani = animation.FuncAnimation(
38     fig=fig,
39     func=update,
40     frames=t,          # Pass the time array to the update
41     function
42     init_func=init,    # Optimize by running init() first
43     blit=True,        # Only redraw parts of the screen
44     that changed
45     interval=50        # Delay between frames in
46     milliseconds
47 )
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
268
```

```

15 PHI = np.linspace(0, 2*np.pi, 30)
16 Theta, Phi = np.meshgrid(THETA, PHI)
17
18 # Spherical to Cartesian parametric conversion
19 X = r * a * np.sin(Theta) * np.cos(Phi)
20 Y = r * b * np.sin(Theta) * np.sin(Phi)
21 Z = r * c * np.cos(Theta)
22
23 # 3. LINEAR ALGEBRA ROTATION OPERATOR (X-Axis)
24 def rotate_x(x, y, z, angle):
25     cos_a = np.cos(angle)
26     sin_a = np.sin(angle)
27
28     # Apply standard R_x(angle) matrix multiplication manually
29     x_new = x
30     y_new = cos_a * y - sin_a * z
31     z_new = sin_a * y + cos_a * z
32
33     return x_new, y_new, z_new
34
35 # 4. UPDATE FUNCTION (3D ENGINE)
36 def update(frame):
37     ax.clear() # Clear previous frame's 3D collection to
38               # prevent memory leaks
39
40     # Restrict axes to maintain spatial consistency
41     ax.set_xlim(-2.5, 2.5)
42     ax.set_ylim(-2.5, 2.5)
43     ax.set_zlim(-2.5, 2.5)
44     ax.set_box_aspect([1,1,1]) # Enforce rigid proportions
45
46     # Calculate current rotational angle
47     angle = frame * 0.05
48
49     # Apply rotation operator to the entire parametric
50     # coordinate mesh
51     Xr, Yr, Zr = rotate_x(X, Y, Z, angle)
52
53     # Render the new coordinates as a 3D wireframe manifold
54     ax.plot_wireframe(Xr, Yr, Zr, color="magenta")
55     ax.set_title(f"Rigid Body Rotation (Ellipsoid)\nAngle={

```

```
        angle:.2f}\_rad")
54
55 # 5. EXECUTE 3D FUNCANIMATION
56 ani = FuncAnimation(
57     fig,
58     update,
59     frames=200,
60     interval=50
61 )
62
63 plt.show()
```

Listing 8.2: Generalized 3D Animation Architecture (Rotating Ellipsoid)

By maintaining this strictly modular approach across both domains, the project successfully abstracts the pure mathematical logic away from the graphical rendering backend, allowing for rapid iteration and prototyping of highly complex geometric systems.

Chapter 9

Results and Analysis

This chapter discusses the output of codes, although we only show static pics here, for gifs and videos, one can visit [GITHUB REPO](#)

9.1 Kinematic Geometry: The Sliding Ladder

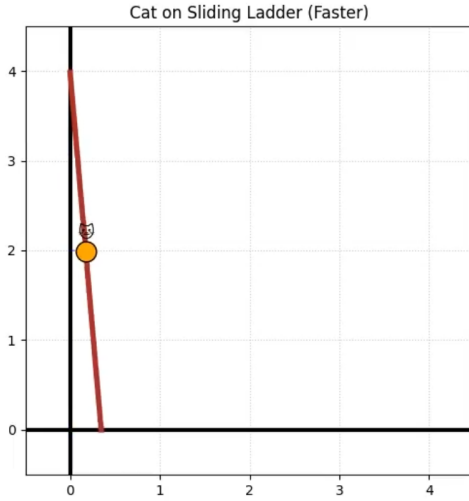
The classical sliding ladder problem was simulated by constraining a line segment of fixed length between the orthogonal axes. As the angle decreases, the locus of a specific point on the ladder (such as the midpoint or the location of a cat) traces a distinct path. The sequence below captures this dynamic progression over four distinct frames.

Interactive Code Available!

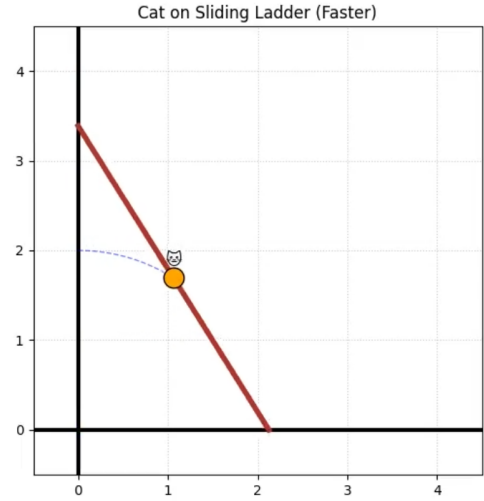
You can view, run, and experiment with the full Python source code for the sliding ladder animation and other geometries in the official GitHub repository:

[Click here to view: cat_at_middle_ladder.py](#)

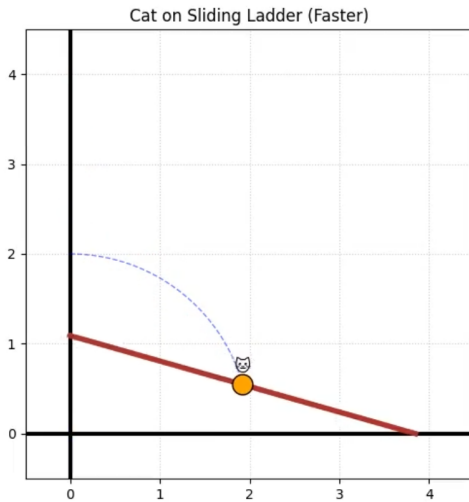
[https://github.com/Shubham18024/Python-Mathematical-Animation/
blob/main/2d-anim/scripts/cat_at_middle_ladder.py](https://github.com/Shubham18024/Python-Mathematical-Animation/blob/main/2d-anim/scripts/cat_at_middle_ladder.py)



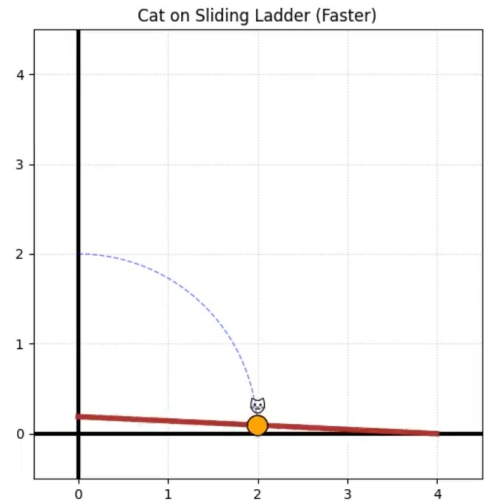
(a) Frame 1: Ladder starts sliding.



(b) Frame 2: Cat's path begins.



(c) Frame 3: Circular arc becomes visible.



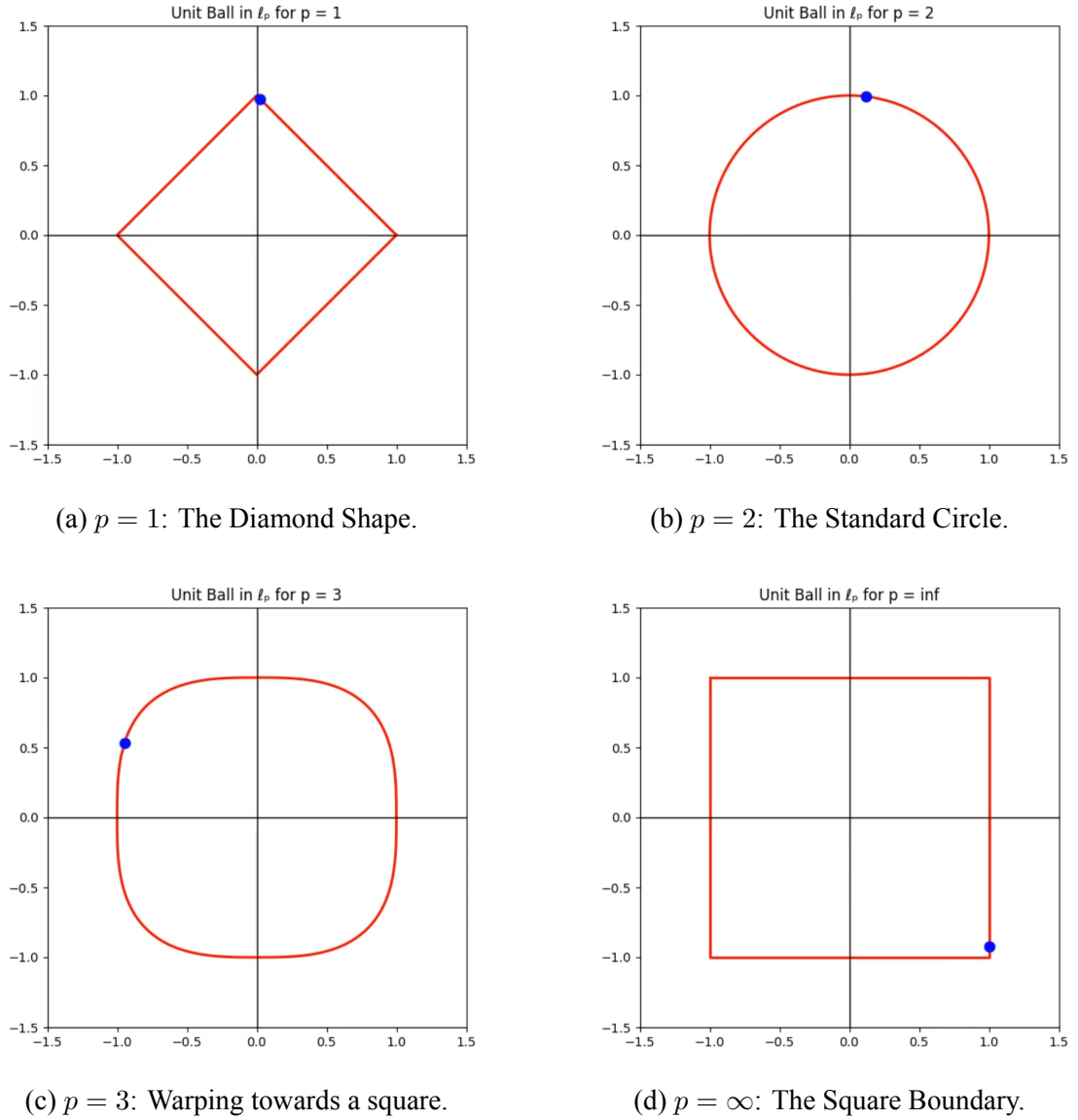
(d) Frame 4: Ladder hits the floor.

Figure 9.1: The Cat on the Sliding Ladder: A visual progression over time.

9.2 Visualizing Metric Spaces: L_p Norms

Beyond classical kinematics, the framework was utilized to visualize abstract functional spaces. The L_p norm defines mathematical "distance." By rendering the unit ball ($\|x\|_p = 1$) for various values of p , we can observe the topological morphing of the space.

As shown in the grid below, the geometry shifts from a sharp diamond ($p = 1$) to a perfect Euclidean circle ($p = 2$), and ultimately converges toward a square boundary ($p = \infty$).

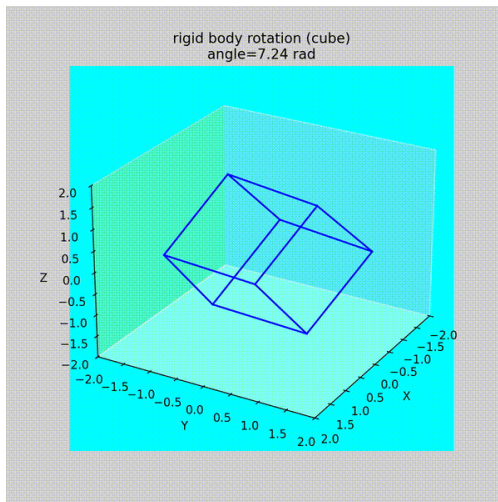
Figure 9.2: Visual progression of the l_p unit ball as p increases.

9.3 3D Surface Geometry and Rigid Body Dynamics

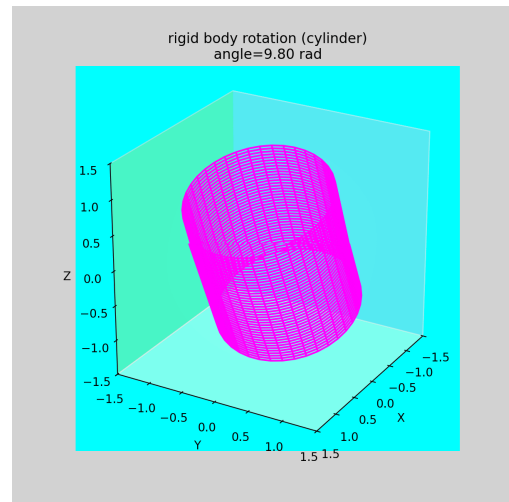
Transitioning from 2D to 3D visualization introduces significant computational complexity. The engine must evaluate bivariate parametric grids and apply $SO(3)$ rotation matrices to every vertex simultaneously. The following sections showcase the successful rendering and animation of various 3D mathematical systems.

9.3.1 Fundamental Rigid Bodies

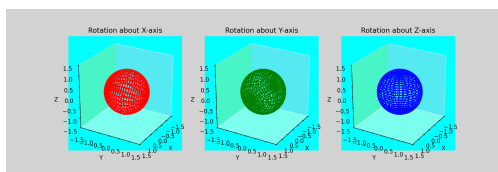
The first test of the 3D engine was to ensure that basic geometric primitives could be rotated without distorting their volume or structural integrity.



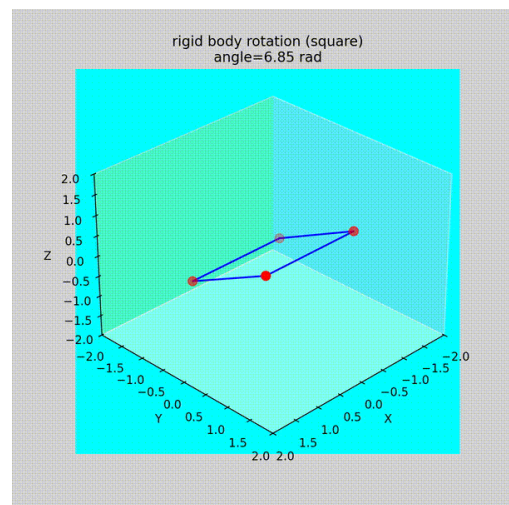
(a) Standard rigid cube rotation.



(b) Parametric cylinder rotation.



(c) Spheres rotating on multiple axes.

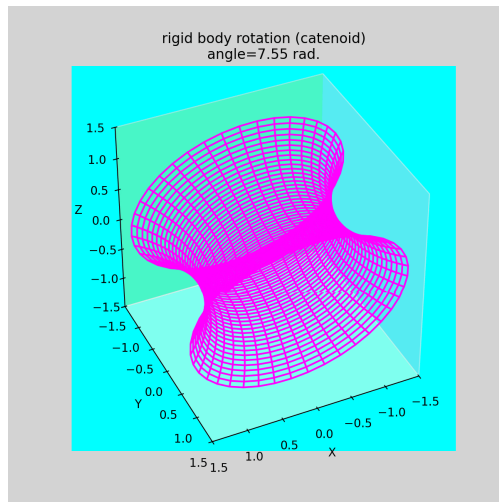


(d) 2D square rotating in 3D space.

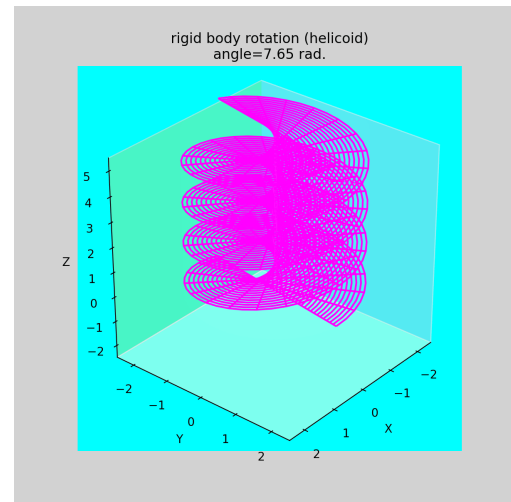
Figure 9.3: Fundamental geometric rigid body rotations.

9.3.2 Complex and Minimal Surfaces

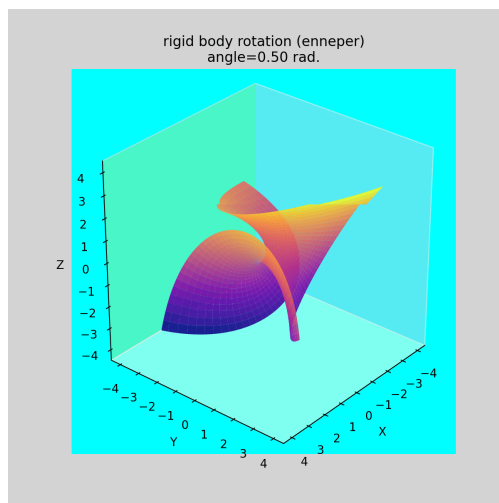
Beyond basic shapes, the engine was used to visualize advanced differential geometry. This includes minimal surfaces (surfaces that locally minimize their area, like soap films) and topological immersions (like Boy's Surface).



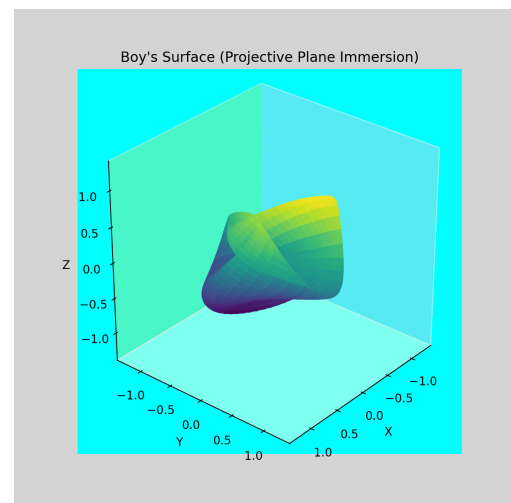
(a) The Catenoid (Minimal Surface).



(b) The Helicoid (Ruled Minimal Surface).



(c) Enneper's Surface (Self-intersecting).

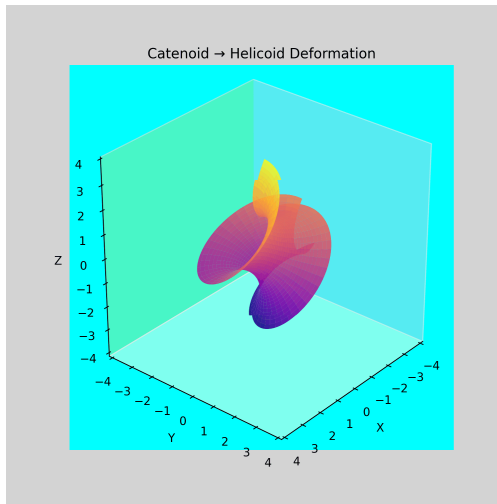


(d) Boy's Surface (RP^2 immersion).

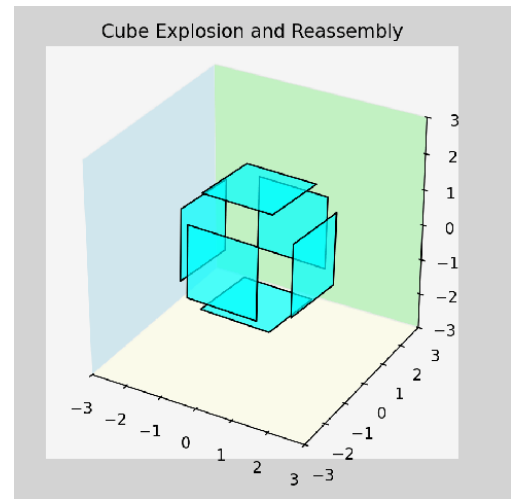
Figure 9.4: Visualization of advanced differential geometry.

9.3.3 Dynamic Systems and Transformations

The most advanced capability of the animation engine is handling multiple moving objects, topological transformations, and combined translation-rotation kinematics.

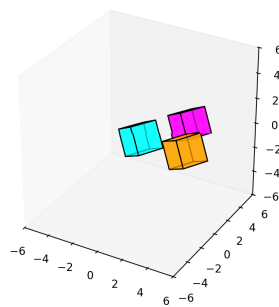


(a) Continuous isometric deformation.



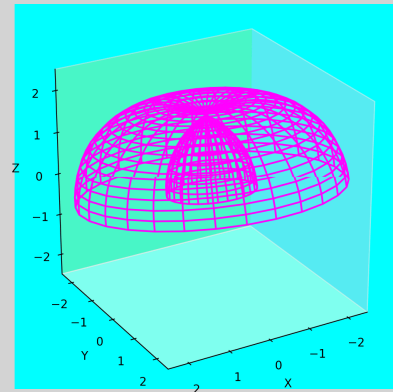
(b) Cube face explosion/reassembly.

Rotation + Translation (Multiple Cubes)



(c) Multi-cube translation and rotation.

rigid body rotation (cutaway spindle torus)
angle=0.15 rad
Rotation about X-axis



(d) Cutaway view of a Spindle Torus.

Figure 9.5: Dynamic multi-body systems and topological transformations.

Simple Human Motion (Articulated Figure)

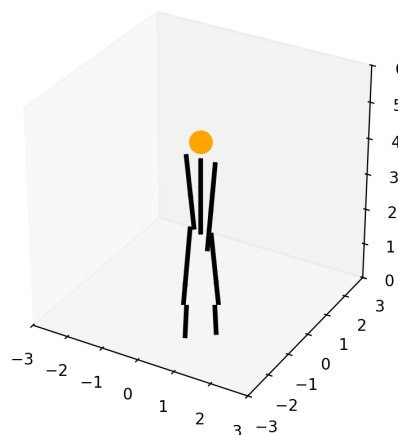


Figure 9.6: Combining multiple geometric arrays to simulate human kinematic motion.

Chapter 10

Challenges Faced

Developing this project and especially a custom mathematical engine from the start-ing that suits your requirement isn't easy. I always tried to bridge the gap between pure abstract geometrical concepts and discrete computation, but throughout this sev-eral technical, mathematical, environmental and system barriers were faced and then also resolved in the organized way.

10.1 Rendering and Performance Issues

The primary graphical backend for this project is `Matplotlib`. While it is exceptional for static data visualization, but pushing it to the render dynamic 3D animations intro-duced immediate performance bottlenecks:

- **CPU-Bound Processing:** Unlike modern graphics engines that utilize the GPU, `Matplotlib` calculates all rendering on the CPU. When animating complex topolo-gies like the Helicoid with dense parametric meshes, the frame rate dropped sig-nificantly, requiring careful optimization of the `np.linspace` resolution.[3]
- **Depth Sorting (Z-buffering) Artifacts:** `Matplotlib`'s 3D axis does not pos-sess a true 3D rendering engine. It relies on a 2D projection using a basic **"painter's algorithm."** [3] This occasionally caused visual glitches where background poly-gons were incorrectly drawn on top of foreground polygons.
- **Memory Leaks in the Animation Loop:** Initially, continuously redrawing ob-jects in the `update()` function caused the RAM usage to spike until the program crashed. This was solved by implementing `ax.clear()` to flush the geometry from memory on every frame before drawing the new coordinates.

10.2 Mathematical Conversion and Scaling

Translating pure mathematics into a visual medium required resolving several geometric scaling issues:

- **Aspect Ratio Distortion:** By default, Python plotting libraries auto-scale axes to fit data. When rotating a 3D sphere, this auto-scaling caused the sphere to stretch into an ellipsoid depending on the viewing angle. Hardcoding `ax.set_box_aspect([1,1,1])` and manually enforcing isometric axis limits was necessary to preserve rigid body dimensions.
- **Boy's Surface Plotting:** Plotting of Minimal surfaces isn't easy especially Boy's Surface, we use its approximate parametrized equation to plot that surface. [6]
- **Continuous to Discrete Mapping:** Mathematical curves extend to infinity, but computers can only draw discrete points. Finding the exact mathematical step-size (e.g., $\Delta\theta = 0.05$) was a constant balancing act between making curves look perfectly smooth and keeping the rendering speed high.

10.3 Environment and Dependency Bugs

Managing a Python environment for graphical rendering across different operating systems introduced "dependency hell":

- **Library Version Conflicts:** Early in development, mismatched versions of NumPy and Matplotlib caused matrix multiplication functions (@ operator) to fail. This is why the project migrated to uv, which completely eliminated these crashes by strictly locking dependency versions in the `pyproject.toml` file.[5]
- **Video Export Failures (FFmpeg):** While displaying the animation on the screen worked perfectly, saving the animations as .mp4 or .gif files required an external binary called FFmpeg. Debugging path variable errors and codec issues within the Linux/Ubuntu environment was a major challenge before the video export pipeline was finally stabilized.[3]

Overcoming these challenges required a deep understanding of both linear algebra and programming skills, ultimately resulting in the highly stable and reproducible animation framework presented in this project work.

Chapter 11

Conclusion and Future Scope

11.1 Summary of Achievements

This project successfully connected pure mathematical theory with computational visualization. By using principles from coordinate geometry, linear algebra, and differential geometry, we developed a strong, open-source Python animation engine.

The project turned static math concepts—like the sliding ladder constraint, $SO(3)$ rotation matrices, and L_p norm spaces—into dynamic, time-dependent computational systems. Additionally, the successful rendering of complex topological shapes, such as Boy’s Surface and the Catenoid-Helicoid transformation, proves that programmatic animation is a valuable tool for modern math education.

11.2 Current Limitations

While the project met its main visualization goals, it is important to recognize the computational limitations of the current setup:

- **CPU-Bound Rendering:** Matplotlib handles 3D rendering on the CPU instead of the GPU. With highly complex shapes that have thousands of vertices, the frame rate drops, making real-time interaction slow.
- **Lack of Real-Time Interactivity:** Right now, animations are pre-calculated and shown as loops or exported videos. Users cannot easily drag a vertex or change a setting through a graphical slider during an animation without editing the source code.
- **Pseudo-3D Projection:** Matplotlib’s 3D axis does not support true depth-buffering (Z-buffering) or advanced lighting methods, which sometimes causes visual issues where background faces appear on top of foreground faces.

11.3 Future Scope

To build on the foundation laid by this project, future development can explore several technical paths:

1. **Migration to High-Performance Graphics Engines:** To address the CPU limitations, future versions of this code could shift the mathematical logic to GPU-accelerated frameworks. Using libraries like `Manim` (designed for math animation) or `Three.js` (for web browsers) would enable rendering complex lighting and true Z-buffering at 60 frames per second.
2. **Graphical User Interface (GUI) Development:** Creating a dedicated application frontend with `PyQt` or `CustomTkinter` would let students enter their own parametric equations, adjust slider values for radius or time, and see the geometry respond in real-time without needing to deal with the raw Python scripts.
3. **Integration of Physics Engines:** Currently, the animations rely solely on geometric rules and time parameters. Future work could add rigid-body physics engines (like `SciPy`'s numerical ODE solvers) to introduce mass, gravity, inertia, and collision detection into the mathematical models.

*"Nature's patterns are the most beautiful,
but they are all governed by the same mathematical laws. In computer animation,
we don't just draw; we simulate the laws of the universe."*

— ED Catmull (Founder of Pixar Studios)

Bibliography

- [1] Waxman, J. (2023). *Cognitive Theory of Multimedia Learning* [Monograph]. Center for Health Decision Science, Harvard T.H. Chan School of Public Health. Available at: https://media.repository.chds.hsph.harvard.edu/static/filer_public/ca/62/ca625803-3d73-4855-b3e1-765870ce3772/2023_jwaxman_monograph_cogtheory_multimed.pdf
- [2] Python Software Foundation. (2024). *Python Language Reference, version 3.x*. Available at: <https://docs.python.org/3/>
- [3] Matplotlib Developers. (2024). *Matplotlib Animation Guide*. Available at: https://matplotlib.org/stable/api/animation_api.html
- [4] NumPy Developers. (2024). *NumPy Official Documentation*. Available at: <https://numpy.org/doc/stable/>
- [5] Astral. (2024). *uv: An extremely fast Python package installer and resolver, written in Rust*. Astral. Available at: <https://github.com/astral-sh/uv>
- [6] Wikipedia contributors. (2026). *Boy's surface*. Wikipedia, The Free Encyclopedia. Available at: https://en.wikipedia.org/wiki/Boy's_surface
- [7] Loney, S. L. (1895). *The Elements of Coordinate Geometry*. London: Macmillan and Co.
- [8] Kreyszig, E. (1989). *Introductory Functional Analysis with Applications*. John Wiley & Sons. Wiley-India Student Edition.
- [9] Friedberg, S. H., Insel, A. J., & Spence, L. E. (2019). *Linear Algebra* (5th ed.). Pearson Education India.
- [10] Kobayashi, S. (2019). *Differential Geometry of Curves and Surfaces*. Springer Nature Singapore. Corrected publication 2021. <https://doi.org/10.1007/978-981-15-1739-6>

Appendix A

Open-Source Code & GitHub Repository

The complete Python source code, including all Jupyter Notebooks, python scripts, and rendered assets discussed in this project, has been made fully open-source.

Official GitHub Repository

<https://github.com/Shubham18024/Python-Mathematical-Animation>

*Readers are highly encouraged to clone the repository, run the interactive notebooks,
and collaborate or contribute to future mathematical animations.*

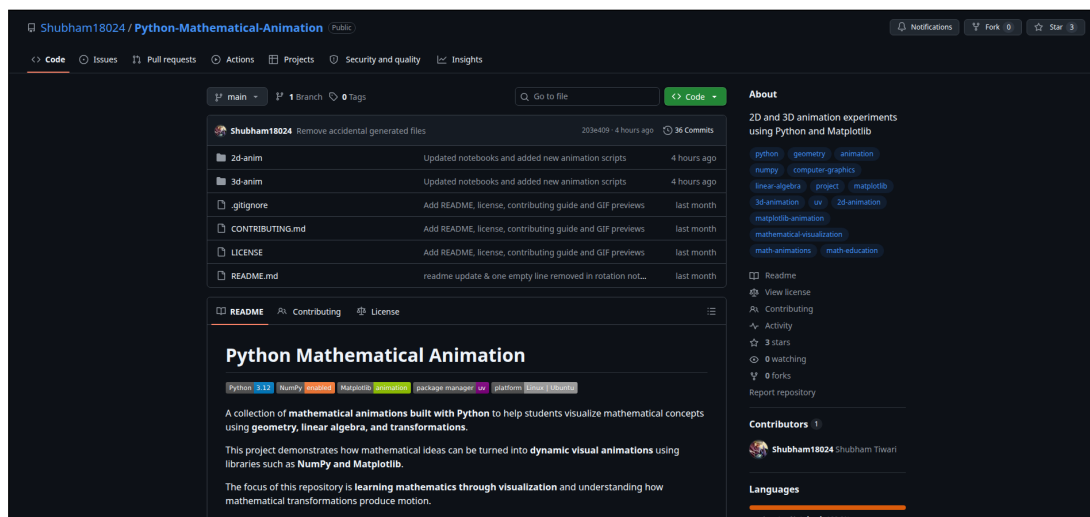


Figure A.1: The project's official GitHub repository landing page.

Appendix B

Proof of Publication

The findings and introductory methodologies of this project have been formally accepted for academic publication. The official acceptance letter from **AkiNik Publications** confirming the publication of the chapter titled "*Introduction: Escaping the Static Chalkboard*" in the edited book *Research Trends in Mathematics and Statistics (Volume - 35)* is attached below.

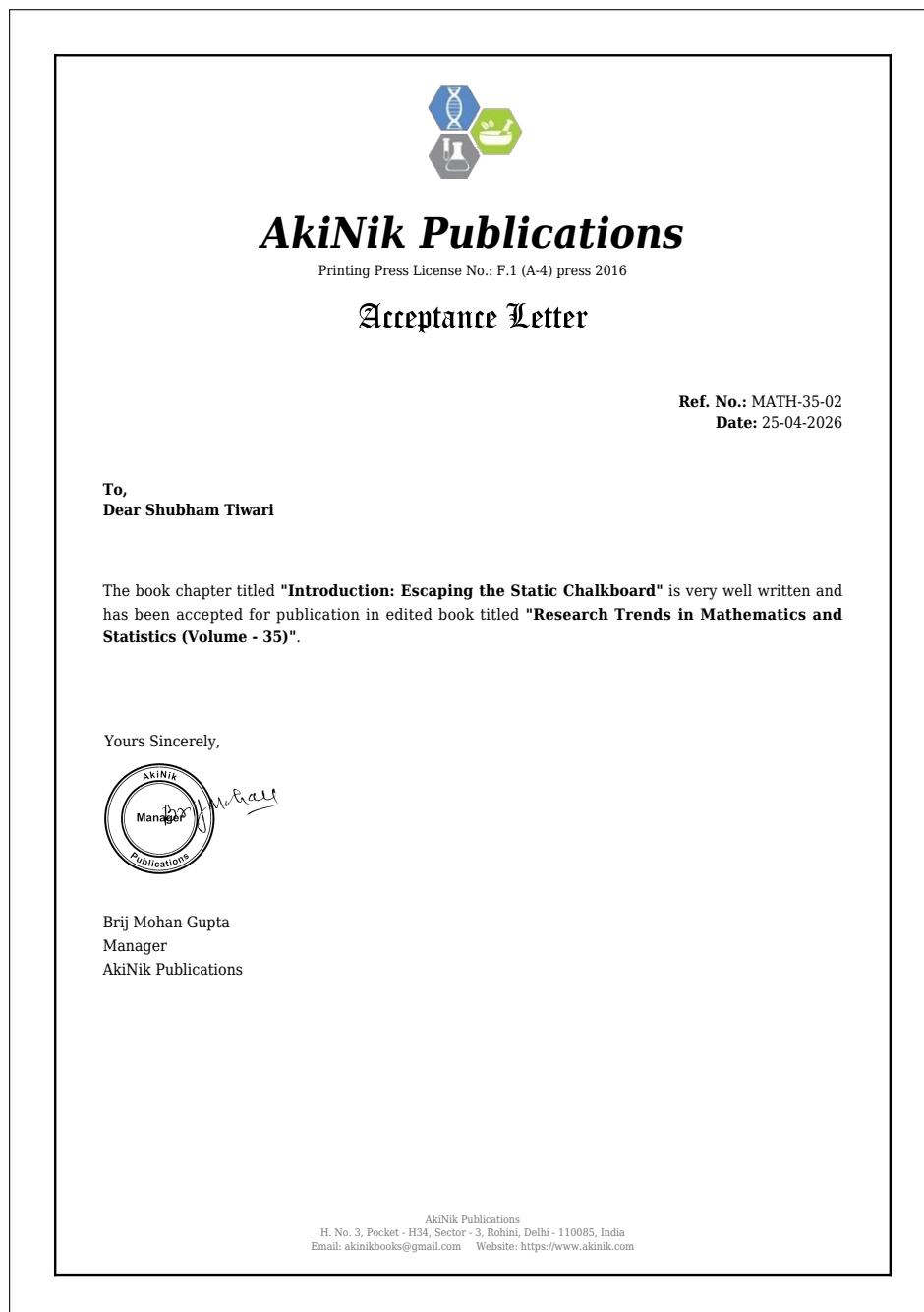


Figure B.1: Official publication acceptance letter from AkiNik Publications (Ref. No.: MATH-35-02).

Appendix C

Conference Presentation Certificate

This project was formally presented at the *International Conference on Scientific Computing and its Applications (SCA-2026)* at South Asian University, and the official certificate of paper presentation is attached below.

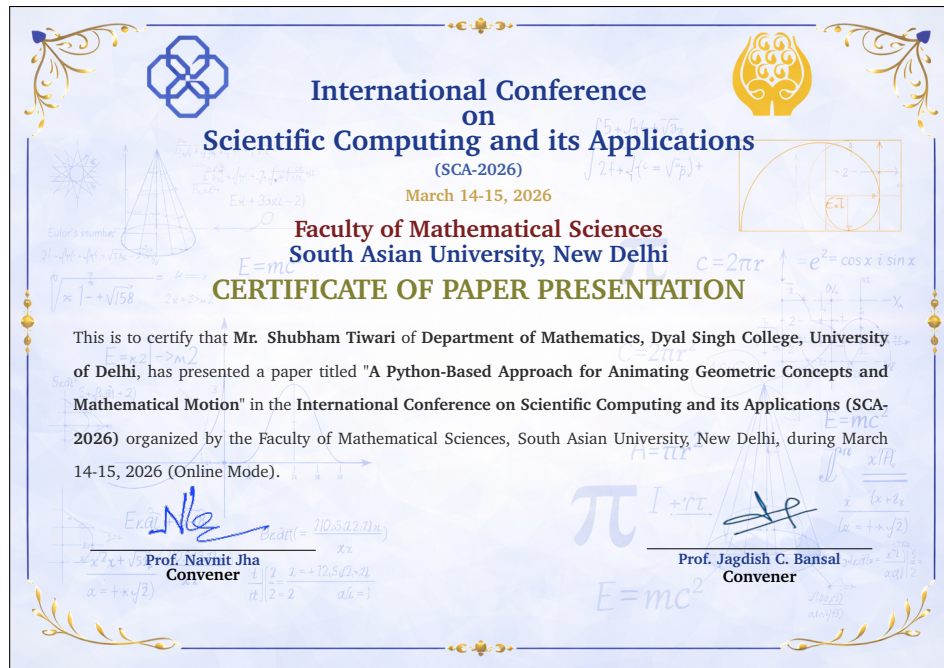


Figure C.1: Official Certificate of Paper Presentation at SCA-2026.

Appendix D

Presentation Slides

This appendix contains selected slides from the presentation of this project.

The Power of Orthogonality in Animation

The orthogonal property $R^T R = I$ (meaning $R^{-1} = R^T$) is computationally profound:

- **Computational Efficiency:** Finding an inverse matrix is usually expensive. For orthogonal rotation matrices, reversing a rotation is just a simple matrix transpose, making frame-by-frame rendering incredibly fast [7].
- **Rigid Body Integrity:** Because length and angles are preserved, 3D models do not stretch, shear, or distort during complex tumbling [1].
- **Numerical Stability:** Repeated matrix multiplications can accumulate floating-point errors. Orthogonal matrices naturally resist degradation over time in continuous simulations.

Shubham Tiwari (Dyal Singh College, UniversA Python based approach to animate games) March 2026 9 / 20

(a) Slide 1: The Power of Orthogonality.

Minimal Surfaces

$$H = 0$$

- Zero mean curvature surfaces [2]
- Arise in optimization and physics
- Example: soap films

Importance:

- Differential Geometry
- Material science
- Computer graphics

Shubham Tiwari (Dyal Singh College, UniversA Python based approach to animate games) March 2026 11 / 20

(b) Slide 2: Minimal Surfaces.

Articulated Motion: Human Movement

- Human motion can be modeled as a system of interconnected points (joints) and line segments (limbs).
- Each joint is represented as a point in $\mathbb{R}^3$, and motion is generated using transformations over time:

$$P_i(t) = R_i(t)P_i + T_i(t)$$

- Complex motion emerges from simple rotational transformations applied hierarchically. [1]
- This forms the basis of **kinematics**, widely used in computer graphics, robotics, and animation.

[Show articulated human motion animation] [7]

Shubham Tiwari (Dyal Singh College, UniversA Python based approach to animate games) March 2026 13 / 20

(c) Slide 3: Human Articulated Motion.

Mathematics Behind DOOM

$$\text{Ray}(t) = P + td$$

- Rays simulate vision
- Distance determines height
- Height $\propto \frac{1}{\text{distance}}$ [3]

3D illusion from purely 2D mathematics

Shubham Tiwari (Dyal Singh College, UniversA Python based approach to animate games) March 2026 15 / 20

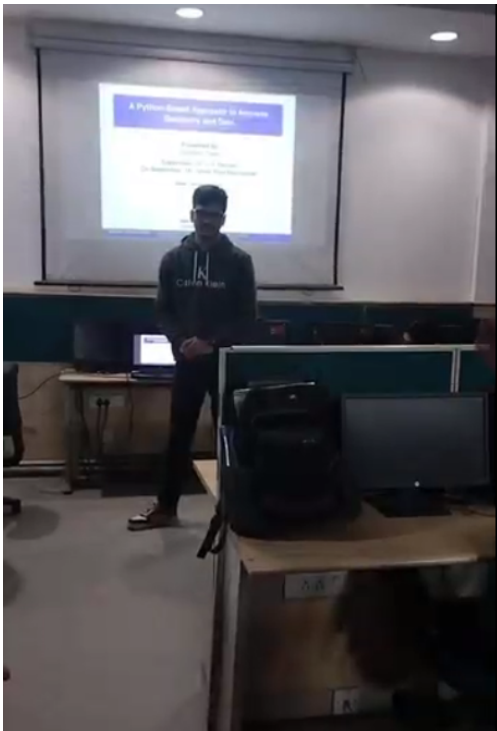
(d) Slide 4: Mathematics behind game 'DOOM'.

Figure D.1: Core slides from the official project presentation.

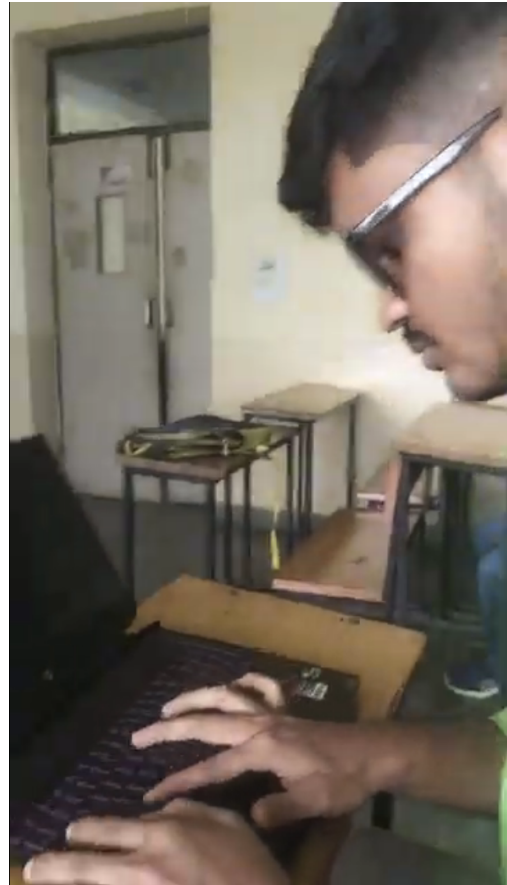
Appendix E

Research Progress & Monthly Reports

To ensure rigorous academic development and continuous peer review, formal Monthly Progress Reports (MPRs) were presented to the project supervisor and co-supervisor throughout Semester VIII. The grid below contains the photographic records of the formal presentations delivered from January 2026 to April 2026.



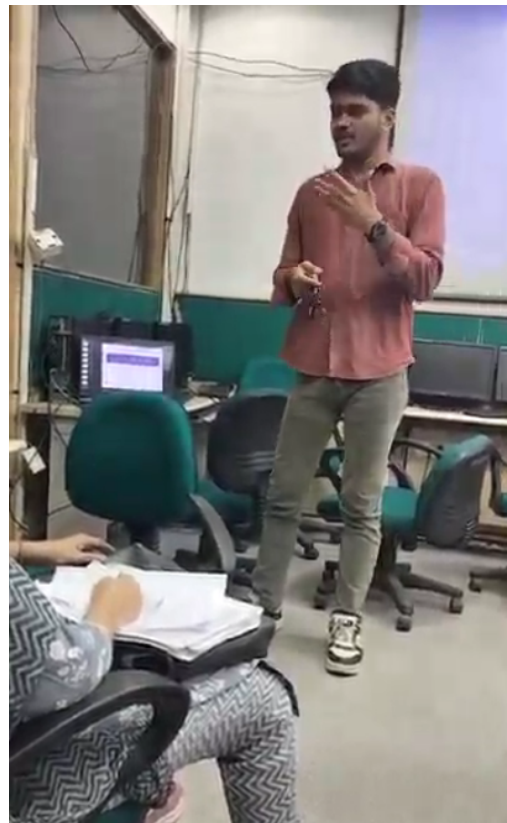
(a) January 2026 Progress Presentation.



(b) February 2026 Progress Presentation.



(c) March 2026 Progress Presentation.



(d) April 2026 Final Review Presentation.

Figure E.1: Monthly progress review presentations delivered during Semester VIII.

Appendix F

Extended Code Snippets

This appendix contains extended utility functions, environmental setups, and comprehensive execution scripts that drive the core animations discussed in this project report. These snippets are provided for reproducibility and deeper technical inspection.

F.1 3D Rigid Body Engine: Rotating Cube

The following script demonstrates the baseline setup for the 3D canvas and the implementation of $SO(3)$ sequential rotation matrices applied to a discrete vertex array (a cube).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4 from IPython.display import HTML
5 plt.rcParams["animation.embed_limit"] = 50 # MB
6
7 def base_setup():
8     fig, ax = plt.subplots(
9         figsize=(6, 6),
10        dpi=100,
11        subplot_kw={"projection": "3d"}
12    )
13
14    ax.set_xlim(-1.5, 1.5)
15    ax.set_ylim(-1.5, 1.5)
16    ax.set_zlim(-1.5, 1.5)
17
18    ax.set_xlabel("X")
19    ax.set_ylabel("Y")
20    ax.set_zlabel("Z")
21
22    ax.set_box_aspect([1,1,1])
23    ax.view_init(elev=25, azim=30)
```

```

24
25     fig.patch.set_facecolor("lightgray") # whole canvas
26     ax.set_facecolor("cyan") # background of 3D boxes
27     ax.xaxis.pane.set_facecolor("lightblue") # background of x-
        axis plane
28     ax.yaxis.pane.set_facecolor("lightgreen") # background of y
        -axis plane
29     ax.zaxis.pane.set_facecolor("lightyellow") # background of
        z-axis plane
30     ax.grid(False) # turn off grid
31
32     return fig, ax
33
34 fig, ax = base_setup()
35
36 # Cube vertices
37 P = np.array([
38     [-1, 1, 1, -1, -1, 1, 1, -1],
39     [-1, -1, 1, 1, -1, -1, 1, 1],
40     [-1, -1, -1, -1, 1, 1, 1, 1]
41 ])
42
43 # Cube edges
44 edges = [
45     (0, 1), (1, 2), (2, 3), (3, 0),
46     (4, 5), (5, 6), (6, 7), (7, 4),
47     (0, 4), (1, 5), (2, 6), (3, 7)
48 ]
49
50 lines = []
51 for e in edges:
52     line, = ax.plot(
53         [P[0, e[0]], P[0, e[1]]],
54         [P[1, e[0]], P[1, e[1]]],
55         [P[2, e[0]], P[2, e[1]]],
56         color='blue'
57     )
58     lines.append(line)
59
60 def Rx(t):
61     return np.array([

```

```

62         [1,0,0],
63         [0,np.cos(t),-np.sin(t)],
64         [0,np.sin(t), np.cos(t)]
65     ])
66
67     def Rz(t):
68         return np.array([
69             [np.cos(t),-np.sin(t),0],
70             [np.sin(t), np.cos(t),0],
71             [0,0,1]
72         ])
73
74     def update(frame):
75         theta = frame * 0.04
76
77         R = Rz(theta) @ Rx(theta)
78         P_rot = R @ P
79
80         for line, edge in zip(lines, edges):
81             i, j = edge
82             line.set_data(
83                 [P_rot[0,i], P_rot[0,j]],
84                 [P_rot[1,i], P_rot[1,j]]
85             )
86             line.set_3d_properties(
87                 [P_rot[2,i], P_rot[2,j]]
88             )
89
90         ax.set_title(f"Rigid Body Rotation (Cube)\nAngle={theta:.2f
91                      }\nrad")
92         return lines
93
94     ani = FuncAnimation(fig, update, frames=200, interval=60, blit=
95         True)
96     plt.close(fig)
97     HTML(ani.to_jshtml())

```

Listing F.1: 3D Cube Rotation using Euler Matrices and FuncAnimation

F.2 Multi-Axis Spherical Rotations

This script utilizes a meshgrid approach to generate parametric spheres and visualizes simultaneous isolated rotations across the X, Y, and Z axes using a multi-subplot 3D architecture.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4 from IPython.display import HTML
5
6 # SPHERE PARAMETERS
7 r = 1
8
9 theta = np.linspace(0, np.pi, 30)
10 phi = np.linspace(0, 2*np.pi, 30)
11
12 Theta, Phi = np.meshgrid(theta, phi)
13
14 # Spherical to Cartesian mapping
15 X = r * np.sin(Theta) * np.cos(Phi)
16 Y = r * np.sin(Theta) * np.sin(Phi)
17 Z = r * np.cos(Theta)
18
19 # Rotation functions
20 def rotate_x(x, y, z, angle):
21     c = np.cos(angle)
22     s = np.sin(angle)
23     x_new = x
24     y_new = c*y - s*z
25     z_new = s*y + c*z
26     return x_new, y_new, z_new
27
28 def rotate_y(x, y, z, angle):
29     c = np.cos(angle)
30     s = np.sin(angle)
31     x_new = c*x + s*z
32     y_new = y
33     z_new = -s*x + c*z
34     return x_new, y_new, z_new
35
36 def rotate_z(x, y, z, angle):

```

```

37     c = np.cos(angle)
38     s = np.sin(angle)
39     x_new = c*x - s*y
40     y_new = s*x + c*y
41     z_new = z
42     return x_new, y_new, z_new
43
44 # FIGURE SETUP (3 SUBPLOTS)
45 fig = plt.figure(figsize=(12,4))
46
47 ax1 = fig.add_subplot(131, projection="3d")
48 ax2 = fig.add_subplot(132, projection="3d")
49 ax3 = fig.add_subplot(133, projection="3d")
50
51 def setup_axis(ax):
52     ax.set_xlim(-1.5,1.5)
53     ax.set_ylim(-1.5,1.5)
54     ax.set_zlim(-1.5,1.5)
55
56     ax.set_box_aspect([1,1,1])
57     ax.grid(False)
58     ax.view_init(elev=25, azim=30)
59
60     ax.set_xlabel("X")
61     ax.set_ylabel("Y")
62     ax.set_zlabel("Z")
63
64     fig.patch.set_facecolor("lightgray")
65     ax.set_facecolor("cyan")
66     ax.xaxis.pane.set_facecolor("lightblue")
67     ax.yaxis.pane.set_facecolor("lightgreen")
68     ax.zaxis.pane.set_facecolor("lightyellow")
69
70 def update(frame):
71     ax1.clear()
72     ax2.clear()
73     ax3.clear()
74
75     setup_axis(ax1)
76     setup_axis(ax2)
77     setup_axis(ax3)

```

```
78
79     angle = frame * 0.05
80
81     # X rotation
82     Xx, Yx, Zx = rotate_x(X, Y, Z, angle)
83     ax1.plot_wireframe(Xx, Yx, Zx, color="red")
84     ax1.set_title("Rotation about X-axis")
85
86     # Y rotation
87     Xy, Yy, Zy = rotate_y(X, Y, Z, angle)
88     ax2.plot_wireframe(Xy, Yy, Zy, color="green")
89     ax2.set_title("Rotation about Y-axis")
90
91     # Z rotation
92     Xz, Yz, Zz = rotate_z(X, Y, Z, angle)
93     ax3.plot_wireframe(Xz, Yz, Zz, color="blue")
94     ax3.set_title("Rotation about Z-axis")
95
96 ani = FuncAnimation(fig, update, frames=200, interval=50)
97
98 plt.close(fig)
99 HTML(ani.to_jshtml())
```

Listing F.2: Simultaneous Multi-Axis Parametric Sphere Rotation

F.3 Topological Geometry: The Cutaway Torus

This script models a Cutaway Horn Torus, illustrating the application of parametric constraints. It demonstrates how to wrap a 2D meshgrid around a circular axis in 3D space, followed by an affine rotation sequence.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4 from IPython.display import HTML
5
6 def base_setup():
7     fig, ax = plt.subplots(figsize=(6, 6), dpi=100, subplot_kw
8         ={"projection": "3d"})
9     # Environment setup abstracted for brevity (see previous
10        scripts)
11     return fig, ax
12
13 fig, ax = base_setup()
14
15 # TORUS PARAMETERS
16 r = 1 # radius of torus tube
17 R = 1 # distance from center to tube center (Horn Torus when r=
18        R)
19
20 THETA = np.linspace(0, np.pi, 30) # polar angle (0 to pi
21        creates the cutaway)
22 PHI = np.linspace(0, 2*np.pi, 30) # azimuthal angle (0 to 2pi)
23
24 Theta, Phi = np.meshgrid(THETA, PHI)
25
26 # SPHERICAL to CARTESIAN
27 X = (R + r * np.cos(Theta)) * np.cos(Phi)
28 Y = (R + r * np.cos(Theta)) * np.sin(Phi)
29 Z = r * np.sin(Theta)
30
31 # ROTATION FUNCTIONS
32 def rotate_x(x, y, z, angle):
33     cos = np.cos(angle)
34     sin = np.sin(angle)
35     return x, cos*y - sin*z, sin*y + cos*z

```

```
33 def update(frame):
34     ax.clear()
35     ax.set_xlim(-2.5,2.5)
36     ax.set_ylim(-2.5,2.5)
37     ax.set_zlim(-2.5,2.5)
38     ax.grid(False)
39     ax.set_box_aspect([1,1,1])
40     ax.view_init(elev=25, azim=60)
41
42     angle = frame * 0.05
43     Xr, Yr, Zr = rotate_x(X, Y, Z, angle)
44
45     ax.plot_wireframe(Xr, Yr, Zr, color="magenta")
46     ax.set_title(f"Rigid□Body□Rotation□(Cutaway□Horn□Torus)\
47                 nAngle={angle:.2f}□rad")
48
49 ani = FuncAnimation(fig, update, frames=200, interval=50)
50 plt.close(fig)
HTML(ani.to_jshtml())
```

Listing F.3: Parametric Modeling and Rotation of a Cutaway Horn Torus

F.4 Differential Topology: Boy's Surface

Boy's surface is a complex, self-intersecting immersion of the real projective plane (RP^2) into R^3 . The code requires the evaluation of advanced spherical harmonic parameterizations to render the surface accurately before applying rigid body rotations.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4 from IPython.display import HTML
5
6 fig, ax = plt.subplots(figsize=(8,8), subplot_kw={"projection":
7     "3d"})
8
9 # PARAMETRIC DOMAIN
10 U = np.linspace(0, np.pi, 80)
11 V = np.linspace(0, 2*np.pi, 80)
12 U, V = np.meshgrid(U, V)
13
14 # BOY SURFACE PARAMETRIZATION
15 X = np.cos(U) * np.sin(V) - np.sin(U) * np.sin(2*V)
16 Y = np.sin(U) * np.sin(V) + np.cos(U) * np.sin(2*V)
17 Z = np.cos(V) + np.cos(2*U)
18
19 # Normalize scale to prevent bounding box clipping
20 scale = np.max(np.abs([X,Y,Z]))
21 X /= scale
22 Y /= scale
23 Z /= scale
24
25 def rotate_y(x, y, z, angle):
26     return np.cos(angle)*x + np.sin(angle)*z, y, -np.sin(angle)*
27         x + np.cos(angle)*z
28
29 def rotate_x(x, y, z, angle):
30     return x, np.cos(angle)*y - np.sin(angle)*z, np.sin(angle)*
31         y + np.cos(angle)*z
32
33 def update(frame):
34     ax.clear()
35     ax.grid(False)
36     ax.set_box_aspect([1,1,1])

```

```

34     ax.view_init(elev=30, azimuth=40)
35
36     angle = frame * 0.05
37
38     # Apply dual-axis rotation
39     Xr, Yr, Zr = rotate_y(X, Y, Z, angle)
40     Xr, Yr, Zr = rotate_x(Xr, Yr, Zr, angle*0.6)
41
42     limit = 1.4
43     ax.set_xlim(-limit, limit)
44     ax.set_ylim(-limit, limit)
45     ax.set_zlim(-limit, limit)
46
47     # Render as a solid continuous surface using colormaps
48     ax.plot_surface(Xr, Yr, Zr, cmap="viridis", edgecolor="none",
49                    alpha=0.9)
50     ax.set_title("Boy's Surface (Projective Plane Immersion)")
51
52 ani = FuncAnimation(fig, update, frames=200, interval=50)
53 plt.close(fig)
54 HTML(ani.to_jshtml())

```

Listing F.4: Mathematical Immersion and Rendering of Boy's Surface

F.5 Combined Kinematics: Translation and Rotation

This script represents the most complex physical modeling in the project, computing translation vectors (T) and rotation matrices (R) simultaneously across multiple independent objects.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4 from mpl_toolkits.mplot3d.art3d import Poly3DCollection
5 from IPython.display import HTML
6
7 fig = plt.figure()
8 ax = fig.add_subplot(111, projection="3d")
9
10 # Cube vertices and topological faces
11 P = np.array([
12     [-1, 1, 1, -1, -1, 1, 1, -1],
13     [-1, -1, 1, 1, -1, -1, 1, 1],
14     [-1, -1, -1, -1, 1, 1, 1, 1]
15 ])
16
17 faces = [
18     [0, 1, 2, 3], [4, 5, 6, 7], [0, 1, 5, 4],
19     [1, 2, 6, 5], [2, 3, 7, 6], [3, 0, 4, 7]
20 ]
21
22 def Rx(t):
23     return np.array([[1, 0, 0], [0, np.cos(t), -np.sin(t)], [0, np.
24         sin(t), np.cos(t)]])
25
26 def Rz(t):
27     return np.array([[np.cos(t), -np.sin(t), 0], [np.sin(t), np.
28         cos(t), 0], [0, 0, 1]])
29
30 def draw_cube(P_transformed, color):
31     face_polygons = []
32     for f in faces:
33         face = [[P_transformed[0, i], P_transformed[1, i],

```

```

34     cube = Poly3DCollection(face_polygons, facecolors=color,
35                             edgecolors="black", alpha=0.7)
36
37     ax.add_collection3d(cube)
38
39     def update(frame):
40         ax.clear()
41         ax.set_xlim(-6,6)
42         ax.set_ylim(-6,6)
43         ax.set_zlim(-6,6)
44         ax.set_box_aspect([1,1,1])
45         ax.grid(False)
46
47         theta = frame * 0.04
48         R = Rz(theta) @ Rx(theta)
49
50         # Cube 1: Stationary Center Rotation
51         P1 = R @ P
52         draw_cube(P1, "cyan")
53
54         # Cube 2: Linear Translation + Rotation
55         tx = np.sin(theta) * 4
56         T1 = np.array([tx,0,0])
57         P2 = R @ P + T1[:,None] # Broadcasting translation vector
58         draw_cube(P2, "orange")
59
60         # Cube 3: Orbital Motion + Rotation
61         tx2 = np.cos(theta) * 4
62         ty2 = np.sin(theta) * 4
63         T2 = np.array([tx2,ty2,0])
64         P3 = R @ P + T2[:,None]
65         draw_cube(P3, "magenta")
66
67         ax.set_title("Rotation_+Translation_(Multiple_Cubes)")
68
69     ani = FuncAnimation(fig, update, frames=200, interval=50)
70     plt.close(fig)
71     HTML(ani.to_jshtml())

```

Listing F.5: Simultaneous Orbit, Translation, and Rotation of Multiple Solids

F.6 Articulated Kinematics: Human Motion Simulation

This script demonstrates hierarchical coordinate transformations. By defining a root node (the hip) and chaining rotation matrices (R_x) down through the limbs, the code successfully simulates a walking bipedal figure using pure vector addition and trigonometric oscillation.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4 from IPython.display import HTML
5
6 fig = plt.figure()
7 ax = fig.add_subplot(111, projection='3d')
8
9 ax.set_xlim(-3,3)
10 ax.set_ylim(-3,3)
11 ax.set_zlim(0,6)
12 ax.set_box_aspect([1,1,1])
13 ax.grid(False)
14
15 # Rotation around x axis (for limb swing)
16 def Rx(theta):
17     return np.array([
18         [1,0,0],
19         [0,np.cos(theta),-np.sin(theta)],
20         [0,np.sin(theta), np.cos(theta)]
21     ])
22
23 # Function to draw limb vectors
24 def draw_limb(start, vec):
25     end = start + vec
26     ax.plot(
27         [start[0],end[0]], [start[1],end[1]], [start[2],end
28         [2]],
29         color="black", linewidth=3
30     )
31     return end
32
33 def update(frame):
34     ax.clear()
35     ax.set_xlim(-3,3)

```

```

35     ax.set_ylim(-3,3)
36     ax.set_zlim(0,6)
37     ax.set_box_aspect([1,1,1])
38     ax.grid(False)
39
40     t = frame * 0.08
41
42     # Body root
43     hip = np.array([0,0,2])
44
45     # Torso and Head
46     torso_vec = np.array([0,0,2])
47     neck = draw_limb(hip, torso_vec)
48     ax.scatter(neck[0], neck[1], neck[2]+0.5, s=200, color="
         orange")
49     shoulder = neck
50
51     # Arm swing (Oscillating)
52     arm_angle = np.sin(t)
53     R_arm = Rx(arm_angle)
54
55     # Left Arm
56     upper_arm = R_arm @ np.array([0,0,-1.2])
57     elbow = draw_limb(shoulder + np.array([0.4,0,0]), upper_arm
        )
58     lower_arm = R_arm @ np.array([0,0,-1])
59     draw_limb(elbow, lower_arm)
60
61     # Right Arm (Inverse Oscillation)
62     R_arm2 = Rx(-arm_angle)
63     upper_arm2 = R_arm2 @ np.array([0,0,-1.2])
64     elbow2 = draw_limb(shoulder + np.array([-0.4,0,0]),
        upper_arm2)
65     lower_arm2 = R_arm2 @ np.array([0,0,-1])
66     draw_limb(elbow2, lower_arm2)
67
68     # Leg swing
69     leg_angle = np.sin(t)
70
71     # Left Leg
72     R_leg = Rx(-leg_angle)

```

```

73     upper_leg = R_leg @ np.array([0,0,-1.5])
74     knee = draw_limb(hip + np.array([0.3,0,0]), upper_leg)
75     lower_leg = R_leg @ np.array([0,0,-1.5])
76     draw_limb(knee, lower_leg)
77
78     # Right Leg
79     R_leg2 = Rx(leg_angle)
80     upper_leg2 = R_leg2 @ np.array([0,0,-1.5])
81     knee2 = draw_limb(hip + np.array([-0.3,0,0]), upper_leg2)
82     lower_leg2 = R_leg2 @ np.array([0,0,-1.5])
83     draw_limb(knee2, lower_leg2)
84
85     ax.set_title("Simple_Human_Motion_(Articulated_Figure)")
86
87 ani = FuncAnimation(fig, update, frames=200, interval=50)
88 plt.close(fig)
89 HTML(ani.to_jshtml())

```

Listing F.6: Hierarchical Kinematics of an Articulated Human Figure

F.7 Stochastic Processes: 2D Random Walk

This script visualizes a classic discrete stochastic process. By utilizing a random number generator to dictate direction, it animates the continuous path of a particle under Brownian-like motion on a 2D Cartesian plane.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 # Setup figure and axes
6 fig, axs = plt.subplots()
7 a = 7
8 axs.set_xlim(-a, a)
9 axs.set_ylim(-a, a)
10 axs.axhline(0, color='black', linewidth=3)
11 axs.axvline(0, color='black', linewidth=3)
12 axs.set_aspect('equal', adjustable='box')
13
14 N = 100 # Total frames
15
16 # Path initialization
17 x, y = [0], [0]
18 path_line, = axs.plot([], [], 'r-', linewidth=2)
19
20 def init():
21     x.clear()
22     y.clear()
23     x.append(0)
24     y.append(0)
25     path_line.set_data([], [])
26     return path_line,
27
28 def update(frame):
29     xold = x[frame]
30     yold = y[frame]
31
32     # Stochastic step logic
33     rnd = np.random.randint(1, 5)
34     if rnd == 1:
35         xnew, ynew = xold + 1, yold
36     elif rnd == 2:

```

```
37         xnew, ynew = xold - 1, yold
38     elif rnd == 3:
39         xnew, ynew = xold, yold + 1
40     else:
41         xnew, ynew = xold, yold - 1
42
43     x.append(xnew)
44     y.append(ynew)
45     path_line.set_data(x, y)
46     return path_line,
47
48 ani = animation.FuncAnimation(fig, update, frames=N, init_func=
    init, blit=True, interval=50)
49
50 plt.title("Random Walk 2D")
51 plt.xlabel("x")
52 plt.ylabel("y")
53 plt.grid(True)
54 plt.show()
```

Listing F.7: Discrete 2D Random Walk Simulation

F.8 Metric Space Topology: L_p Norm Unit Balls

This script dynamically visualizes the morphing topology of unit balls in $\mathbb{R}^2$ under different L_p norms. It includes a specific conditional block to handle the topological limit when $p = \infty$ (the Chebyshev distance).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 def setup_axes():
6     fig, ax = plt.subplots(figsize=(6,6))
7     ax.set_xlim(-1.5, 1.5)
8     ax.set_ylim(-1.5, 1.5)
9     ax.axhline(0, color="black", linewidth=1)
10    ax.axvline(0, color="black", linewidth=1)
11    ax.set_aspect("equal")
12    ax.set_title("Unit_Ball_in_Different_Norms_on_R^2")
13    return fig, ax
14
15 def lp_unit_ball(p, num_points=400):
16     theta = np.linspace(0, 2*np.pi, num_points)
17
18     # Limit Condition for p = infinity (Chebyshev distance)
19     if p == np.inf:
20         n = num_points // 4
21         x = np.concatenate([
22             np.linspace(-1, 1, n),
23             np.ones(n),
24             np.linspace(1, -1, n),
25             -np.ones(n)
26         ])
27         y = np.concatenate([
28             np.ones(n),
29             np.linspace(1, -1, n),
30             -np.ones(n),
31             np.linspace(-1, 1, n)
32         ])
33         return x[:num_points], y[:num_points]
34
35     # Standard parametric generation for finite p
36     x = np.sign(np.cos(theta)) * (np.abs(np.cos(theta)) ** (2/p))

```

```

    ))
37     y = np.sign(np.sin(theta)) * (np.abs(np.sin(theta)) ** (2/p
    ))
38     return x, y
39
40 def update(frame):
41     ax.clear()
42
43     p_index = frame // points_per_norm
44     t_index = frame % points_per_norm
45     current_p = p_values[p_index]
46
47     x, y = lp_unit_ball(current_p)
48     ax.plot(x, y, 'r', linewidth=2)
49
50     ax.set_title(f"Unit_Ball_in_L_p_for_p={current_p}")
51     ax.set_xlim(-1.5, 1.5)
52     ax.set_ylim(-1.5, 1.5)
53     ax.set_aspect("equal")
54     ax.axhline(0, color="black", linewidth=1)
55     ax.axvline(0, color="black", linewidth=1)
56
57     ax.plot(x[t_index], y[t_index], 'bo', markersize=8)
58     return []
59
60 p_values = [1,2,3,4,5,6,7,8,9,10,np.inf]
61 points_per_norm = 300
62 total_frames = len(p_values) * points_per_norm
63
64 fig, ax = setup_axes()
65 ani = animation.FuncAnimation(fig, update, frames=total_frames,
    interval=20)
66 plt.show()

```

Listing F.8: Visualizing L_p Unit Balls from $p = 1$ to $p = \infty$

Appendix G

Plagiarism Originality Report

shubham_acad_report_v1

by J.v.ramani Shubham

Submission date: 01-May-2026 04:54PM (UTC+0700)

Submission ID: 2949659859

File name: shubham_acad_report_v1.pdf (6.89M)

Word count: 6768

Character count: 68188

shubham_acad_report_v1

ORIGINALITY REPORT

2%

SIMILARITY INDEX

2%

INTERNET SOURCES

1%

PUBLICATIONS

1%

STUDENT PAPERS

PRIMARY SOURCES

1

archive.org

Internet Source

1%

2

opus.bibliothek.uni-wuerzburg.de

Internet Source

1%

3

David J. Pine. "Introduction to Python for
Science and Engineering", CRC Press, 2019

Publication

<1%

4

pagaian.org

Internet Source

<1%

5

Submitted to University of Southampton

Student Paper

<1%

Exclude quotes On

Exclude matches < 14 words

Exclude bibliography On

Appendix H

AI Writing Overview



Shubham J.v.ramani

shubham_acad_report_v1

- Quick Submit
- Quick Submit
- College Users IQAC, Library, Others

Document Details

Submission ID

trn:oid:::1:3556888401

Submission Date

May 1, 2026, 4:53 PM GMT+7

Download Date

May 1, 2026, 5:07 PM GMT+7

File Name

shubham_acad_report_v1.pdf

File Size

6.9 MB

78 Pages

6,768 Words

68,188 Characters



0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



0 AI-generated only 0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

